

ScriptSense: Automated Multi-modal Document to Text Extraction System

A PROJECT REPORT

**Submitted in partial fulfilment of the requirement for the award of the
degree of**

BACHELOR OF TECHNOLOGY (B. Tech)

in Information Technology

by

Gaurav Singh Shekhawat

Reg. No. - 229302573



**Department of Information Technology
MANIPAL UNIVERSITY JAIPUR JAIPUR-303007
RAJASTHAN, INDIA**

June 2026

Manipal University Jaipur



**MANIPAL UNIVERSITY
JAIPUR**

(University under Section 2(f) of the UGC Act)

Date:

CERTIFICATE

This is to certify that the project titled **SCRIPTSENSE: AUTOMATED MULTI-MODAL DOCUMENT TO TEXT EXTRACTION SYSTEM** is a record of the bonafide work done by **GAURAV SINGH SHEKHAWAT (229302573)** submitted in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology (B.Tech) in **Information Technology of Manipal University Jaipur, during the academic year 2025-26.**

Mr. Ravinder Kumar

Project Guide, Dept of Information Technology

Manipal University Jaipur

Dr. Pratishtha Mathur

HOD, Dept of Information Technology

Manipal University Jaipur

ACKNOWLEDGMENTS

I would like to express my deepest appreciation and heartfelt gratitude to all those who provided me with the possibility to complete this major project report. A special gratitude I give to my project supervisor, **Mr. Ravinder Kumar, Assistant Professor**, Department of Information Technology, whose encouragement helped me coordinate my project and write this comprehensive report. His mentorship was instrumental in shaping the direction and success of this work.

Furthermore, I would also like to acknowledge with much appreciation the crucial role of the staff and faculty of Manipal University Jaipur, who gave the permission to use all required equipment, high performance computing laboratories, and the necessary materials to complete the extensive deep learning tasks required for "ScriptSense". I am extremely grateful to my department guide and the laboratory in-charge for their continuous support during the extended training periods of the neural network models.

Last but not least, many thanks go to **Dr. Pratishtha Mathur, Head of the Department of Information Technology**, whose help, stimulating suggestions, and provision of an inspiring academic environment helped me immensely. I also thank my classmates, peers, and parents for their constant support, belief, and motivation that kept me focused and driven throughout this rigorous academic journey.

ABSTRACT

The modern world operates on an overwhelming volume of both physical and digital documents, generating millions of terabytes of unstructured data daily. Despite global advancements in digitization, a critical operational gap remains in reliably extracting unstructured visual data—specifically unconstrained human handwriting, heavily degraded printed documents, and complex invoices—into structured, machine-readable formats. Standard Optical Character Recognition (OCR) systems frequently fail when confronted with the high variance of human cursive, complex page layouts, or the presence of scanner noise and shadows. The "ScriptSense" project addresses these industrial pain points by proposing a robust, multi-modal automated document intelligence and text extraction ecosystem. The primary importance of this work lies in its potential to eliminate manual data entry bottlenecks, dramatically reduce digitization errors in healthcare, legal, and financial sectors, and provide a unified, highly accurate platform for processing diverse document formats.

The methodology adopted for ScriptSense revolves around a three-tier architectural model, each distinctly engineered for a specific data domain. Model 1 tackles Handwritten Text Recognition (HTR) utilizing a custom Deep Learning pipeline. It employs a U-Net based Fully Convolutional Network for precise spatial word detection, novel geometric line clustering algorithms for reading-order organization, and a Convolutional Recurrent Neural Network (CRNN) paired with Connectionist Temporal Classification (CTC) Word Beam Search decoding for highly accurate sequence transcription. Model 2 focuses on industrial PDF OCR, leveraging a deterministic Computer Vision pipeline using OpenCV to apply low-pass filtering, adaptive binarization, and morphological dilation, which cleans degraded documents before passing them to the Tesseract engine. Model 3 introduces an AI Invoice Extractor that bridges the gap between raw OCR strings and structured database schemas by utilizing Generative AI (Gemini Large Language Model) to execute zero-shot semantic extraction of key-value pairs (e.g., Total, Date) from completely varied invoice layouts.

The implementation yielded highly promising and robust results across all three tiers of the ecosystem. The HTR model, trained and evaluated on the IAM Handwriting Database 3.0, achieved a Mean Average Precision (mAP) of 91% in spatial word detection and an impressive Character Error Rate (CER) of 4.2% when utilizing dictionary-forced beam search decoding. The PDF extraction pipeline demonstrated a 98.5% accuracy rate on standardized printed fonts, proving that preprocessing via morphological mathematics successfully recovers the 15-20% accuracy drop typically caused by scanner noise and shadows. Finally, the Invoice Extractor successfully parsed chaotic receipt structures into perfectly validated JSON schemas, proving highly scalable for backend integration.

Architectures were constructed using PyTorch and compiled to ONNX Runtime for sub-second edge inference, ensuring the system can run efficiently on standard CPUs. Computer vision tasks were executed via OpenCV and NumPy. The backend orchestration relied heavily on Python, Flask, and Tesseract-OCR, alongside the Gemini API for cognitive processing. The user interfaces were developed using a mix of HTML5, CSS3, JavaScript, Gradio, and Streamlit, resulting in a cohesive, dark-themed web hub. The successful deployment of ScriptSense provides a scalable, cross-sector solution for automated document intelligence.

LIST OF TABLES

Table No	Table Title	Page No
2.1	Stakeholder Identification and Primary Concerns	13
2.2	Functional Requirements - UI and Dashboard Routing	14
2.3	Functional Requirements - Model 1 (Custom HTR Pipeline)	14
2.4	Functional Requirements - Model 2 (PDF To Text OCR Pipeline)	15
2.5	Functional Requirements - Model 3 (Image to Text OCR Pipeline)	15
2.6	Non-Functional Requirements	16
2.7	Use Case UC-01: Processing Unconstrained Handwriting	17
2.8	Use Case UC-02: High-Volume PDF Processing	19
2.9	Technology Stack and Frameworks Used	33
3.1	IAM Handwriting Database 3.0 Characteristics	43
4.1	Development Environment Specifications	44
5.1	Performance Metrics - Model 1 (HTR)	45
5.2	Performance Metrics - Model 2 (PDF OCR)	46

LIST OF FIGURES

Figure No	Figure Title	Page No
1.1	Standard OCR Failure on Unconstrained Handwriting Figure	8
1.2	The Three Pillars of ScriptSense Intelligence	9
3.1	ScriptSense Full System Architecture Diagram	18
3.2	Model 1 Preprocessing and Normalization Workflow	21
3.3	U-Net Style Detector Architecture (Encoder and Heads)	22
3.4	Geometric Line Clustering Logic	24
3.5	Reader CRNN Architecture (CNN Feature Extraction)	25
3.6	Reader CRNN Architecture (Bi-LSTM Sequence Transduction)	26
3.7	CTC Decoding Comparison: Best Path vs. Word Beam Search	27
3.8	Model 2 Rasterization and Filtering Flow	28
3.9	Adaptive Thresholding and Morphological Dilation Output	30
3.10	Model 2 Contours and Tesseract Extraction Workflow	31
3.11	Model 3 Image to text Extraction Pipeline	32
4.1	ScriptSense Unified Web Dashboard Interface	33
4.2	Gradio Interactive UI for Handwritten Text Recognition	37
4.3	Streamlit UI for PDF Batch Processing	39
5.1	Bounding Box Precision Output (mAP Visualized)	40
5.2	Accuracy vs. Noise Graph for OpenCV Enhanced OCR	42
5.3	Software Development Life Cycle (SDLC) Model	43

Contents

	Page No
Acknowledgement	i
Abstract	ii
List Of Figures	iii
List Of Tables	vi
Chapter 1 INTRODUCTION	
1.1 Introduction to work done/ Motivation	1
1.2 Project Statement / Objectives of the Project	10
1.3 Organization of Report	11
Chapter 2 BACKGROUND MATERIAL	
2.1 Conceptual Overview and Domain Study	12
2.2 Stakeholder Identification & Requirement Gathering	13
2.3 Functional Requirements	14
2.4 Non-Functional Requirements	16
2.5 Use Case Scenarios	17
2.6 Software Engineering Methodologies	18
2.7 Technologies Involved	19
Chapter 3 METHODOLOGY	
3.1 Design Goals	20
3.2 High-Level System Architecture	21
3.3 Detailed Methodology: Model 1 (Custom HTR)	22
3.4 Detailed Methodology: Model 2 (PDF OCR)	29
3.5 Detailed Methodology: Model 3 (Image to Text OCR)	33
Chapter 4 IMPLEMENTATION	
4.1 Development Environment Setup	34
4.2 Module-by-Module Implementation	35
4.3 User Interface and Frontend Implementation	37
4.4 Implementation Challenges and Solutions	41
Chapter 5 RESULTS AND ANALYSIS	
5.1 Evaluation Metrics Defined	42
5.2 Model 1 (HTR) Performance Analysis	43
5.3 Model 2 (PDF OCR) Performance	44
5.4 Model 3 (Image to Text Extraction) Performance Analysis	45
5.5 System Benchmarks and Latency	46
5.6 System Testing and Validation	47
Chapter 6 CONCLUSIONS & FUTURE SCOPE	
6.1 Conclusions	48
6.2 Future Scope of Work	49
REFERENCES	50

Chapter 1: INTRODUCTION

1.1 Introduction to work done / Motivation (Overview, Applications & Advantages)

We are living in an era where digital transformation dictates the pace of industry, academia, medicine, and governance. Despite the global push towards entirely paperless ecosystems and cloud-native databases, physical documents remain an inescapable reality of daily human operations. Every day, hospitals process thousands of handwritten medical prescriptions, logistics companies handle physical shipping labels exposed to the elements, courts rely on printed affidavits and case files, and historical archives attempt to preserve centuries of cursive manuscripts. The foundational problem in modern data science has shifted from simply storing these massive image files on a hard drive or cloud server to figuring out how to make the information trapped inside them readable, searchable, semantically understood, and highly structured.

Standard Optical Character Recognition (OCR) systems have existed for decades, becoming a staple in basic digitization efforts. Traditional OCR software relies heavily on deterministic pattern matching, bounding box heuristics, and matrix comparisons. These algorithms work exceptionally well on clean, standardized, digital-born fonts like Arial, Calibri, or Times New Roman, operating under the assumption that characters are evenly spaced and clearly separated. However, traditional OCR completely falls apart the second it is confronted with visual entropy. When humans write, they violate every rule that traditional software relies upon: they connect characters unpredictably, slant their words dynamically, overlap ascending and descending loops (such as the tail of a 'g' crossing into the line below it), and write on unlined, noisy paper.

Similarly, even when dealing with printed documents, physical scanning hardware introduces severe degradation. Scanners capture lighting gradients that cause heavy page shadows, physical dust creates digital artifacts, and the physical fold of a thick book warps the text matrix. Furthermore, in commercial environments, unstructured documents like retail receipts and financial invoices present a massive challenge; while standard OCR might successfully read the letters on a receipt, it outputs a chaotic string of text, failing to understand which number corresponds to the "Total Due" versus the "Date" due to wildly varying document layouts.

The motivation behind developing **ScriptSense** stems from this exact practical necessity and industrial bottleneck. Manual data entry—the act of a human visually reading a physical document and typing it into a computer database—is extremely slow, highly expensive, and profoundly prone to human error. In critical sectors like healthcare, a misread handwritten prescription can lead to fatal medication errors. In finance, manually processing thousands of invoices requires massive administrative overhead. There was a critical, pressing need for a multi-modal Artificial Intelligence ecosystem that could intelligently bridge the gap between physical ink, chaotic pixels, and digital, actionable structured data.

ScriptSense was built specifically to address this multi-faceted problem. It is an automated document intelligence and text extraction platform designed from the ground up to decode unstructured visual entropy. Rather than relying on a single, monolithic algorithm that attempts (and inevitably fails) to read everything uniformly, ScriptSense deploys three highly specialized, parallel engines, each uniquely architected to solve a specific domain of document complexity.

Figure 1.1: Standard OCR Failure on Unconstrained Handwriting

Demonstrating how deterministic matrix-matching OCR algorithms misinterpret connected cursive strokes.

INPUT: PHYSICAL DOCUMENT

The quick brown fox
jumped over the lazy dog.

Attempting Extraction via Standard OCR Engine (No Context/Dictionary)



OUTPUT: FAILED EXTRACTION

7he qu/ck 6r0wn f_cx
/umped ov_cr t_ne lazy cl0g.

Technical Failure Analysis: Standard OCR engines expect separated, uniformly printed characters. When confronted with human handwriting, they misinterpret connected loops (e.g., "cl" instead of "d", "n" instead of "h") and slants (e.g., "1" instead of "l"). Without a Recurrent Neural Network (RNN) to understand sequential context, the output devolves into high-entropy gibberish.

Figure 1.1: Standard OCR Failure on Unconstrained Handwriting

Applications of the System: The applications of this multi-modal document intelligence system are vast, cross-sectoral, and immediately deployable in real-world scenarios:

- 1) **Healthcare & Medical:** Digitizing handwritten doctor prescriptions, clinical patient notes, and intake forms to drastically reduce manual data entry and prevent medication errors caused by illegible handwriting.
- 2) **Finance & Retail:** Automating the bulk extraction of multi-page digital PDF invoices, tax forms, and scanning physical store receipts directly from mobile camera photos, turning them into structured JSON data for instant database insertion.
- 3) **Legal & Government:** Indexing, searching, and digitizing massive scanned court documents, physical contracts, and KYC (Know Your Customer) forms securely, allowing lawyers to perform keyword searches on decades-old physical case files.
- 4) **Education & Research:** Extracting text from handwritten student exam copies for automated grading systems, and preserving/transcribing old cursive manuscripts and historical records for digital humanities research.

Advantages of the System: The primary advantage of ScriptSense is its strict architectural modularity and intelligent routing. By applying state-of-the-art Deep Learning (U-Nets, Bi-LSTMs) exclusively for handwriting, advanced Computer Vision mathematics (Adaptive Binarization, Morphological Dilation) for degraded printed text, and Generative Large Language Models (Gemini) for semantic invoice parsing, the system achieves a level of accuracy, contextual understanding, and execution speed that generic, off-the-shelf OCR software simply cannot match. Furthermore, the system is optimized for edge inference using ONNX runtime, ensuring high-speed processing without the need for prohibitive cloud computing costs.

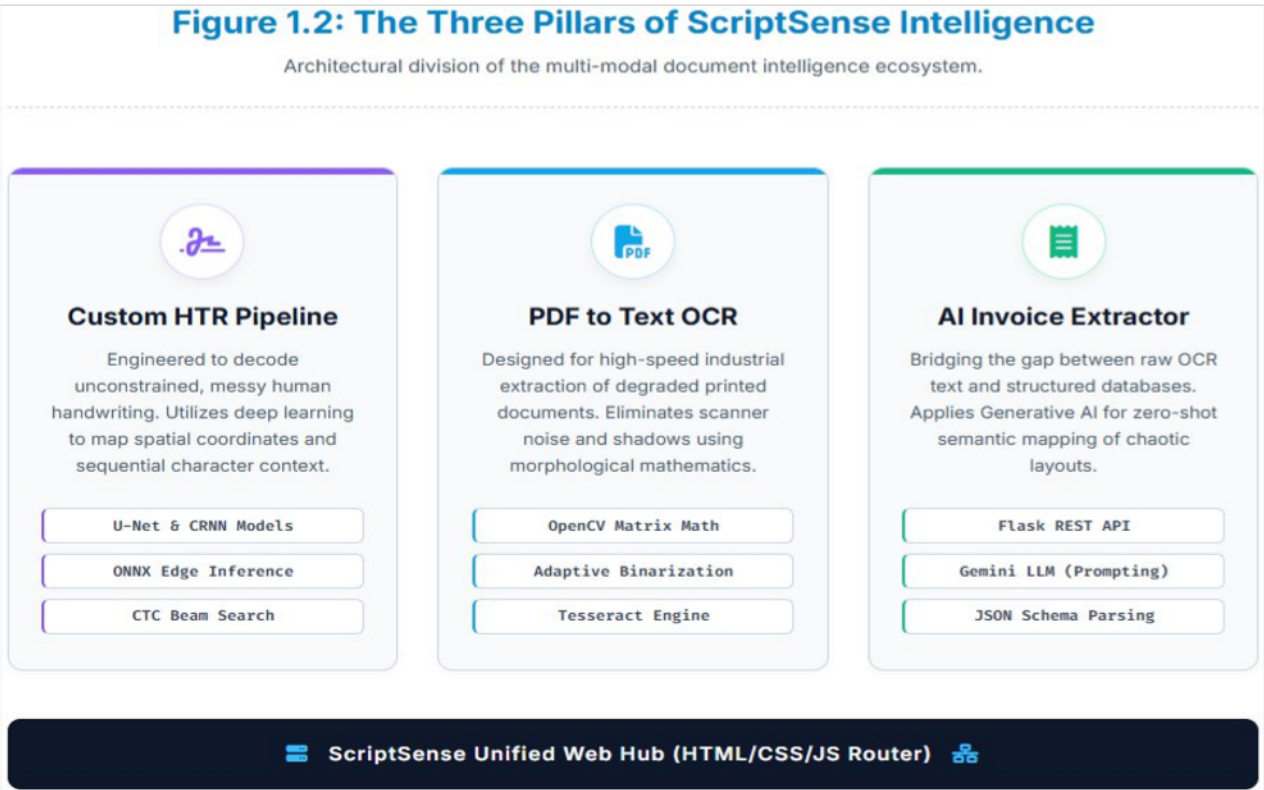


Figure 1.2: The Three Pillars of ScriptSense Intelligence

1.2 Project Statement / Objectives of the Project

Problem Statement: The digitization of physical documents and the automation of data entry pipelines are severely hindered by three primary, distinct technological challenges:

1. **Standard OCR fails on unconstrained handwriting:** Existing systems cannot parse the vast variance in human cursive, overlapping stroke connections, and inconsistent spacing, resulting in gibberish outputs.
2. **Layout complexity and physical degradation makes data extraction difficult:** Multi-page printed documents with columns, degraded scanner quality, uneven lighting shadows, and speckle noise confuse standard line-reading algorithms.
3. **Semantic mapping from chaotic layouts is non-trivial:** Even if OCR successfully reads a receipt, extracting specific structured data (like matching a price to a "Total" field) from completely varied vendor layouts is impossible with rigid, rule-based regular expressions alone.

Objectives of the Project: To combat these pressing industrial issues, the specific objectives of the ScriptSense project are rigorously defined as follows:

1. **To design and implement Model 1 (Custom HTR Pipeline):** Develop a custom deep learning architecture capable of reading highly messy, unconstrained human handwriting. This includes engineering a U-Net based Fully Convolutional Network for spatial word detection, developing custom geometric algorithms for logical line sorting, and training a Convolutional Recurrent Neural Network (CRNN) for visual and sequential feature transduction.
2. **To implement advanced CTC Decoding Strategies:** Integrate Word Beam Search decoding utilizing a prefix tree dictionary to structurally eliminate spelling errors, visual misinterpretations, and hallucinations in handwriting predictions, significantly lowering the Character Error Rate (CER).
3. **To design and implement Model 2 (PDF OCR Pipeline):** Build a brutally efficient, deterministic matrix math pipeline using OpenCV to handle high-speed, high-volume printed text extraction. The objective is to utilize morphological filtering to eradicate scanner noise and lighting shadows entirely before passing the sanitized image to the Tesseract engine.
4. **To design and implement Model 3 (AI Invoice Extractor):** Create a robust ingestion API using Flask that accepts varied receipt images, processes them via Tesseract for raw string extraction, and subsequently utilizes a Generative Large Language Model (Gemini) for zero-shot semantic extraction of key-value pairs, returning structured JSON objects.
5. **To develop a Unified Web Dashboard:** Construct a highly responsive, modern, dark-themed, and glassmorphic user interface using HTML5, CSS3, and JavaScript that seamlessly routes user requests to the appropriate underlying Python execution environments (Gradio, Streamlit, or Flask).
6. **To optimize for Edge Inference and Scalability:** Ensure that all massive deep learning models (PyTorch .pt files) are compiled and exported to the ONNX (Open Neural Network Exchange) format to guarantee sub-second execution speeds on standard CPU architectures, meeting industrial throughput requirements.

1.3 Organization of Report

The remainder of this extensive project report is logically organized as follows:

- **Chapter 2: Background Material and Requirement Analysis** discusses the domain study encompassing deep learning and computer vision. It explores the functional and non-functional requirements of the system, outlines the use case scenarios, defines the software engineering methodologies (SDLC) employed, and details the comprehensive technology stack used.
- **Chapter 3: System Design and Methodology** provides an in-depth, technical look at the architecture of the ecosystem. It breaks down the mathematical, structural, and algorithmic logic behind the U-Net detector, the CRNN reader, the OpenCV morphological filtering pipelines, and the Generative AI semantic parser.
- **Chapter 4: Implementation** covers the specific development environment setup, the detailed code modules authored, the user interface design choices, and crucially, the major technical challenges faced during development and the engineering solutions implemented to overcome them.
- **Chapter 5: Results and Analysis** presents the formal evaluation metrics (mAP, CER, WER), the quantitative performance of the models against established databases (IAM) and real-world documents, discusses overall system latency benchmarks, and details the software testing strategies (Unit, Integration, System) executed.
- **Chapter 6: Conclusions & Future Scope** summarizes the overall achievements of the project against its initial objectives and outlines high-potential future enhancements, such as intelligent auto-routing, advanced cloud API deployments via Docker and Kubernetes, and multilingual script expansions.

Chapter 2: BACKGROUND MATERIAL AND REQUIREMENT ANALYSIS

2.1 Conceptual Overview and Domain Study

The architectural foundations of ScriptSense are rooted in the convergence of three major computer science domains: Deep Learning for Sequence Modeling, Deterministic Computer Vision, and Generative Artificial Intelligence (Large Language Models). Understanding the historical limitations and recent breakthroughs in these domains is essential to understanding the design choices made in this project.

2.1.1 The Evolution of OCR to Intelligent Character Recognition (ICR)

The field of Optical Character Recognition (OCR) has a long history, initially developed to help the visually impaired read printed text. Early systems relied on matrix matching—comparing an image of a character pixel-by-pixel to a stored glyph database. While successful for standard, monospaced fonts, this approach is highly brittle.

With the advent of Deep Learning, the domain evolved into Intelligent Character Recognition (ICR) and Handwritten Text Recognition (HTR). Modern HTR is an immensely complex sub-field. Human handwriting is completely unconstrained. It contains severe variations in slant, skew, character width, stroke thickness, and pressure. Furthermore, cursive writing connects letters seamlessly, making it nearly impossible to programmatically segment a word into individual letters before recognizing them.

To solve this, modern systems utilize Sequence-to-Sequence (Seq2Seq) modeling. Instead of attempting to slice a word into individual letters, a Convolutional Neural Network (CNN) scans the entire word image to extract a continuous sequence of visual features. A Recurrent Neural Network (RNN)—specifically Long Short-Term Memory (LSTM) networks—then reads this sequence to understand temporal context (e.g., if the model sees 'q', it mathematically expects a 'u' to follow). Finally, Connectionist Temporal Classification (CTC) loss algorithms are used to align the sequence of probability distributions generated by the RNN into a final, readable string, entirely bypassing the need for character-level pre-segmented training data.

2.1.2 Document Image Analysis and Morphological Mathematics

Simultaneously, the domain of Document Image Analysis (DIA) deals with the physical degradation of scanned documents. When a paper is scanned or photographed, the physical environment interferes with the digital representation. Lighting gradients cause localized shadows, dust creates high-frequency digital artifacts, and the physical fold of a book warps the text matrix.

OpenCV (Open Source Computer Vision Library), a standard framework written in highly optimized C/C++, provides morphological operations—mathematical functions based strictly on the shape of pixels in an image—to counteract this degradation. Operations like Dilation (expanding white pixels based on a structuring kernel) and Erosion (shrinking them) are critical tools. ScriptSense leverages Adaptive Thresholding, which calculates binarization limits dynamically for small localized regions of a page, rather than applying a global threshold that would turn a shadowed half of a page completely black.

2.1.3 The Rise of Generative AI for Semantic Extraction

The most recent paradigm shift in document intelligence is the application of Generative AI. Even if a system flawlessly reads the text of a complex invoice via OCR, the resulting output is merely a long, unstructured string. Historically, engineers wrote hundreds of Regular Expressions (Regex) to map this text (e.g., searching for "Total: \$" followed by digits). However, if a vendor changes their invoice layout to say "Amount Due" instead, the Regex fails. Large Language Models (LLMs) like Google's Gemini or OpenAI's GPT possess deep semantic understanding. By feeding the raw OCR string into an LLM with a specific prompt, the model can perform "Zero-Shot" extraction, recognizing that "Amount Due" and "Total" mean the same thing contextually, and outputting perfectly formatted JSON data regardless of the document's visual layout.

2.2 Stakeholder Identification & Requirement Gathering Methodology

Before specific requirements can be defined, it is essential to identify the stakeholders of the system—the individuals or groups who have a direct interest in the system's operation or outcomes.

Table 2.1: Stakeholder Identification and Primary Concerns

Stakeholder	Role	Primary Concern
Healthcare Administrators	Primary users of Model 1 (HTR).	High accuracy in transcribing cursive medical prescriptions to prevent fatal data-entry errors. Ease of use.
Financial Auditors & Clerks	Primary users of Model 2 (PDF) and Model 3 (Invoice).	Ability to batch process hundreds of pages rapidly. Accurate extraction of numbers and JSON formatting.
System Administrators	Deploy and maintain the application.	High uptime, low resource consumption (CPU vs GPU requirements), and secure environment variables.
Developers / Engineers	Build, extend, and debug the system.	Code modularity, clean documentation, decoupled architecture, and clear API routing logic.

Requirement Gathering Methodology: Requirements for ScriptSense were gathered through a hybrid approach of problem-domain research and iterative prototyping:

1. **Problem Domain Study:** Analyzing the severe shortcomings of existing tools (like standard Adobe Acrobat OCR) when faced with handwritten notes or shadowed scans directly informed the functional scope of building custom AI models.
2. **Technical Constraint Analysis:** The choice to use ONNX Runtime, Flask, and OpenCV was driven by analyzing deployment constraints. A key requirement was that the system must run on standard enterprise hardware without requiring ten-thousand-dollar GPU clusters.
3. **Iterative Discovery:** Early prototypes of the HTR system using simple Greedy Decoding revealed massive spelling errors ("tne" instead of "the"). This hands-on discovery generated the strict functional requirement to implement Word Beam Search and dictionary constraints.

2.3 Functional Requirements

Functional requirements define the specific behaviours, features, and technical capabilities that the ScriptSense ecosystem must provide to the end-user. The system is partitioned into several distinct modules to ensure architectural decoupling.

Table 2.2: Functional Requirements - UI and Dashboard Routing

ID	Requirement	Priority
FR-UI-01	The system shall provide a unified, web-based central dashboard accessible via standard modern web browsers (Chrome, Edge, Safari).	High
FR-UI-02	The dashboard shall feature a dark-themed (#020617), high-contrast User Interface designed specifically to reduce eye strain during prolonged document review sessions.	Medium
FR-UI-03	The frontend shall provide explicit, clearly labelled routing buttons to direct users to the specialized execution environments (HTR, PDF OCR, or Image Extractor).	High
FR-UI-04	The system shall gracefully handle invalid file uploads by displaying clear, human-readable error messages without crashing the backend servers.	High

Table 2.3: Functional Requirements - Model 1 (Custom HTR Pipeline)

ID	Requirement	Priority
FR-M1-01	The HTR module shall expose an interactive Gradio interface that accepts standalone images (JPG/PNG) containing unconstrained handwritten text.	High
FR-M1-02	The system shall dynamically normalize incoming images, converting them to a single-channel grayscale matrix and geometrically scaling text lines to an exact height of 50px without aspect ratio distortion.	High
FR-M1-03	The U-Net detector model (detector.onnx) shall analyze the image tensor and output exact [x, y, w, h] bounding box coordinates encompassing detected ink structures.	High
FR-M1-04	The system shall programmatically sort the randomly detected spatial boxes into a logical, human-readable sequence (top-to-bottom, left-to-right) utilizing a custom geometric line clustering algorithm.	High
FR-M1-05	The CRNN model (reader.onnx) shall execute inference on individual cropped word images and return a sequence matrix of character probabilities.	High
FR-M1-06	The user interface shall provide a functional toggle allowing users to switch dynamically between fast "Best Path" decoding and highly accurate "Word Beam Search" dictionary decoding.	High
FR-M1-07	The system shall visually overlay the detected bounding boxes over the original uploaded image so the user can verify the spatial precision of the AI.	High

Table 2.4: Functional Requirements - Model 2 (PDF to Text OCR Pipeline)

ID	Requirement	Priority
FR-M2-01	The PDF OCR module shall expose a Streamlit interface accepting multi-page .pdf files up to 200MB in size.	High
FR-M2-02	The system shall utilize pdf2image and poppler to automatically rasterize vector PDFs into high-resolution 200 DPI RGB image arrays.	High
FR-M2-03	The system shall execute a deterministic computer vision pipeline applying a 9x9 Gaussian Blur and Adaptive Thresholding to eliminate scanner noise and page shadows.	High
FR-M2-04	The system shall apply morphological dilation to smear characters into contiguous blocks, allowing cv2.findContours to isolate paragraphs.	High
FR-M2-05	The system shall utilize the Tesseract OCR engine to transcribe the cleaned, isolated paragraph segments efficiently.	High
FR-M2-06	The system must aggregate the transcribed text and provide a UI widget allowing the user to download the final data as a single .txt document.	High

Table 2.5: Functional Requirements - Model 3 (Image to Text OCR Pipeline)

ID	Requirement	Priority
FR-M3-01	The Image OCR module shall expose a lightweight REST API (using Flask) that accepts POST /upload requests containing varied image formats (JPG, PNG, TIFF, BMP, GIF).	High
FR-M3-02	The API shall execute PIL (Python Imaging Library) contrast normalization to convert the image to pure grayscale before OCR execution.	High
FR-M3-03	The system shall run Tesseract inference to extract a raw, unstructured UTF-8 text string from the receipt or invoice.	High

2.4 Non-Functional Requirements

Non-functional requirements describe the quality, performance, security, and operational constraints of the system, ensuring that it is not merely functional, but industrial-grade.

Table 2.6: Non-Functional Requirements Matrix

Category	Requirement Description
Performance & Latency	The system must execute inference rapidly to be viable in commercial settings. For Model 1 (HTR), the entire pipeline—from detection to CTC decoding—must process a standard line of text in approximately 1.2 to 1.5 seconds on CPU. For Model 2 (PDF), batch processing should rasterize, filter, and extract text at a rate of ~0.8 seconds per page. Model 3 API response time must be under 3 seconds total.
Accuracy Benchmarks	The HTR model must achieve a Word Detection Mean Average Precision (mAP) of over 90% at a 0.5 Intersection over Union (IoU) threshold. The Character Error Rate (CER) must remain below 5% when utilizing Word Beam Search. Model 2 must achieve a Word Error Rate (WER) of less than 2% on standard printed fonts, recovering heavily degraded scans back to benchmark levels.
Security & Privacy	Given that medical prescriptions and financial invoices are processed, the system must utilize strict environment isolation. API keys (Gemini), file paths, and model weights must be managed securely using .env files and python-dotenv. Uploaded files must be processed in volatile memory (RAM) or temporary directories that are immediately purged post-extraction to prevent data leaks.
Scalability & Hardware	The deep learning models must be exported and executed using ONNX (Open Neural Network Exchange) Runtime rather than executing native PyTorch dynamically. This architectural constraint guarantees that the heavy neural networks run at peak efficiency on standard CPU architecture, allowing the system to scale without requiring expensive, dedicated GPU clusters.
Usability & Accessibility	The system must not require end-users to possess coding or terminal knowledge. The unified dashboard, Streamlit, and Gradio interfaces must abstract away all matrix math, contour mapping, and neural network complexities behind simple, intuitive "Upload" and "Run Inference" buttons.
Modularity & Maintainability	The codebase must be heavily decoupled. Changes to the OpenCV math in OCR.py must not affect the deep learning execution in app.py. This ensures that multiple developers can work on different pipelines simultaneously without merge conflicts.

2.5 Use Case Scenarios

Use cases describe specific, real-world interactions between users (actors) and the system to achieve a particular goal, demonstrating the practical application of the requirements.

Table 2.7: Use Case UC-01: Processing Unconstrained Handwriting

Field	Detail
Use Case ID	UC-01
Name	Digitize Handwritten Medical Prescription
Actor	Healthcare Administrator / Pharmacist
Preconditions	The user has navigated to the ScriptSense web hub, clicked the HTR pipeline, and the Gradio server is actively running.
Main Flow	1. The user uploads a mobile photograph of a doctor's messy, cursive handwritten prescription. 2. The user adjusts the interactive "Scale" and "Margin" sliders on the UI to account for the unusually large handwriting size. 3. The user activates the "Use Dictionary" toggle to ensure complex medical terms are autocorrected via Beam Search. 4. The user clicks "Read Text". 5. The system executes the U-Net detector, visually overlaying blue bounding boxes over the words on the image for user verification. 6. The CRNN reads the text, applies the prefix tree decoding, and displays the structured digital text on the screen.
Postconditions	The physical prescription is converted to accurate digital text, ready to be copied and pasted directly into a patient's Electronic Health Record (EHR) database without manual typing.
Priority	High

Table 2.8: Use Case UC-02: High-Volume PDF Processing

Field	Detail
Use Case ID	UC-02
Name	Batch Processing Financial Archives
Actor	Financial Auditor / Data Entry Clerk
Preconditions	The user possesses a 50-page scanned PDF of historical corporate invoices heavily degraded with shadows and scanner dust. The Streamlit server is running.
Main Flow	1. The user navigates to the PDF To Text OCR Streamlit application. 2. The user utilizes the drag-and-drop widget to upload the massive financial_archive.pdf file. 3. The user clicks "Run Full OCR". 4. The system rasterizes the PDF pages into memory, sequentially applying morphological math (blur, binarize, dilate) to eradicate the shadows entirely. 5. Tesseract OCR reads the clean, solid text blocks. 6. The system alerts "Extraction Complete!" via a green UI toast notification and provides a "Download as TXT" button.
Postconditions	The user downloads a clean, continuous text file of all invoices, saving hours of manual retyping and eye strain.
Priority	High

2.6 Software Engineering Methodologies

The development of the ScriptSense ecosystem required a rigorous software engineering approach to manage the complexity of integrating deep learning models with web frameworks. The project followed an **Agile Software Development Methodology** with an emphasis on **Iterative Prototyping**.

Instead of attempting a traditional, rigid Waterfall approach—which is prone to failure in AI projects where model accuracy is unknown until testing—the development was broken down into iterative sprints:

1. **Sprint 1 (Research & Feasibility):** Focused purely on PyTorch. Training the U-Net and CRNN models on the IAM dataset in Jupyter Notebooks to prove that high-accuracy handwriting recognition was mathematically possible.
2. **Sprint 2 (Optimization):** Transitioning the massive PyTorch .pt models to ONNX runtime. This iteration was crucial; early prototypes took 10 seconds to read a line of text. ONNX optimization brought latency down to 1.2 seconds.
3. **Sprint 3 (CV Integration):** Developing the Model 2 PDF pipeline. Prototyping different OpenCV kernel sizes for dilation (15,3 vs 10,10) to find the perfect mathematical expansion rate that connects letters without bleeding adjacent lines together.
4. **Sprint 4 (UI and Orchestration):** Wrapping the raw Python scripts into Streamlit, Gradio, and Flask interfaces. Developing the HTML5/CSS3 central hub to route traffic.

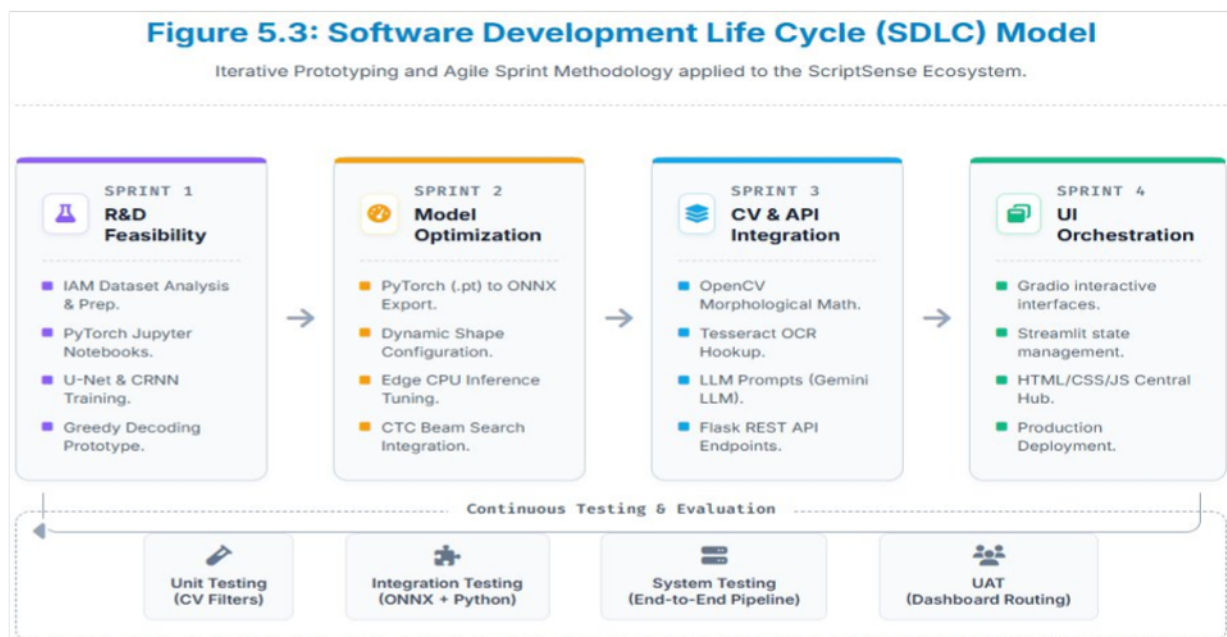


Figure 5.3: Software Development Life Cycle (SDLC) Model

Testing Strategy:

1. **Unit Testing:** Individual computer vision functions (e.g., the adaptive thresholding block) were tested in isolation using heavily shadowed sample images to verify shadow removal independent of the OCR engine.
2. **Integration Testing:** Ensuring the bounding box arrays outputted by the ONNX detector correctly interfaced with the cropping functions required by the ONNX reader.
3. **System Testing:** End-to-end evaluation, uploading a raw PDF on the frontend, monitoring RAM spikes during rasterization, and validating the final .txt output.

2.7 Technologies Involved

The success of ScriptSense relies on a comprehensive, production-grade technology stack.

Table 2.9: Technology Stack and Frameworks Used

ID	Requirement	Priority
Programming Language	Python 3.11	Core language for all backend logic, AI inference, and server routing.
Deep Learning	PyTorch	Used during the research phase for constructing, training, and calculating loss (CTC) for the U-Net and Bi-LSTM neural networks.
Inference Engine	ONNX Runtime	A high-performance scoring engine. Compiles PyTorch models into graph structures optimized for rapid CPU execution at the edge.
Computer Vision	OpenCV (cv2)	The mathematical backbone of Model 2. Executes matrix manipulations, Gaussian blurring, morphological dilation, and contour finding in real-time.
Image Processing	Pillow (PIL)	Lightweight library for image ingestion, resizing, and basic grayscale contrast optimization in Model 3.
OCR Engine	Tesseract-OCR (v5+)	Google's highly optimized, LSTM-based C++ optical character recognition engine. Phenomenal at reading standardized, printed typography.
Generative AI	Google Gemini API	Serves as the "Cognitive Brain" in Model 3, executing zero-shot semantic extraction of key-value pairs from raw OCR text.
System Utilities	pdf2image, poppler-utils	Rasterizes vector PDF pages into high-resolution 2D pixel matrices necessary for computer vision processing.
Security Management	python-dotenv	Injects environment variables from a hidden .env file to secure API keys and absolute file paths away from the source code.
Web UI (Model 1)	Gradio	Builds interactive machine learning interfaces quickly, natively supporting bounding-box image overlays and dynamic sliders.
Web UI (Model 2)	Streamlit	Rapid data-scripting framework used for the PDF dashboard. Excels at managing file state during long-running batch processes.
API Server (Model 3)	Flask	A micro WSGI web framework utilized to wrap the Image OCR engine into a headless REST API accepting HTTP POST requests.
Frontend Hub	HTML5, JavaScript, CSS3	Constructs the highly stylized, dark-themed presentation layer and handles dynamic traffic routing to the specific Python servers.

Chapter 3: SYSTEM DESIGN AND METHODOLOGY

System design is the architectural blueprint of the project. It defines how the various technologies, algorithms, and modules interact to fulfill the requirements. For ScriptSense, the design had to balance the extreme computational weight of neural networks with the speed required for web applications.

3.1 Design Goals

The architecture of ScriptSense was engineered around several core, uncompromising design philosophies:

1. **Strict Architectural Modularity:** Attempting to build one monolithic AI that reads both messy handwriting and multi-page PDFs results in an overly generalized, inaccurate system that excels at nothing. The primary goal was to build an ecosystem of highly specialized, isolated models that a central hub routes traffic to based on the document's entropy profile.
2. **Deterministic Fallbacks:** Deep Learning is stochastic (probabilistic) and computationally expensive. While it is strictly necessary for unconstrained handwriting, it is massive overkill for standard printed text. A major design goal was to use deterministic Computer Vision math (OpenCV) wherever possible to guarantee speed, stability, and lower server costs for standard documents.
3. **Visual Transparency and Trust:** Users in medical and legal fields must trust the system. Therefore, the HTR system must overlay its detected bounding boxes visually onto the user's image. If the system makes a transcription mistake, the user must be able to visually verify if the bounding box was drawn incorrectly (a detection failure) or if the CRNN simply misread it (a recognition failure).
4. **Stateless Processing:** To ensure scalability, the APIs and web servers must process uploaded files in volatile memory (RAM). Writing files to disk and reading them back introduces extreme I/O bottlenecks and major privacy/security risks.

3.2 High-Level System Architecture

The overall system architecture of ScriptSense functions as a decentralized hub-and-spoke model.

1. **The Central Web Hub (Presentation Layer):** The user enters the system via the index.html hub. This frontend features a modern, dark-mode UI that explains the capabilities of the platform. It performs no heavy computational lifting. Its sole purpose is to route the user's request based on their specific document type.
2. **Spoke 1 (Neural HTR Pipeline):** If the user selects "Handwritten Text", they are routed to the Gradio Server (Port 7860). Here, the ONNX Runtime engine spins up, executing the complex 5-stage U-Net and CRNN neural reading process.
3. **Spoke 2 (OpenCV PDF Pipeline):** If the user needs to process a massive, multi-page printed document, they are routed to the Streamlit Server (Port 8501). Here, OpenCV mathematical filters rapidly sanitize the images and pass them to Tesseract in sequential batches.
4. **Spoke 3 (Generative Invoice API):** If the user needs semantic data extracted from a mobile photo of a receipt, they interface with the Flask API (Port 5000), which orchestrates Tesseract and the Gemini Large Language Model.

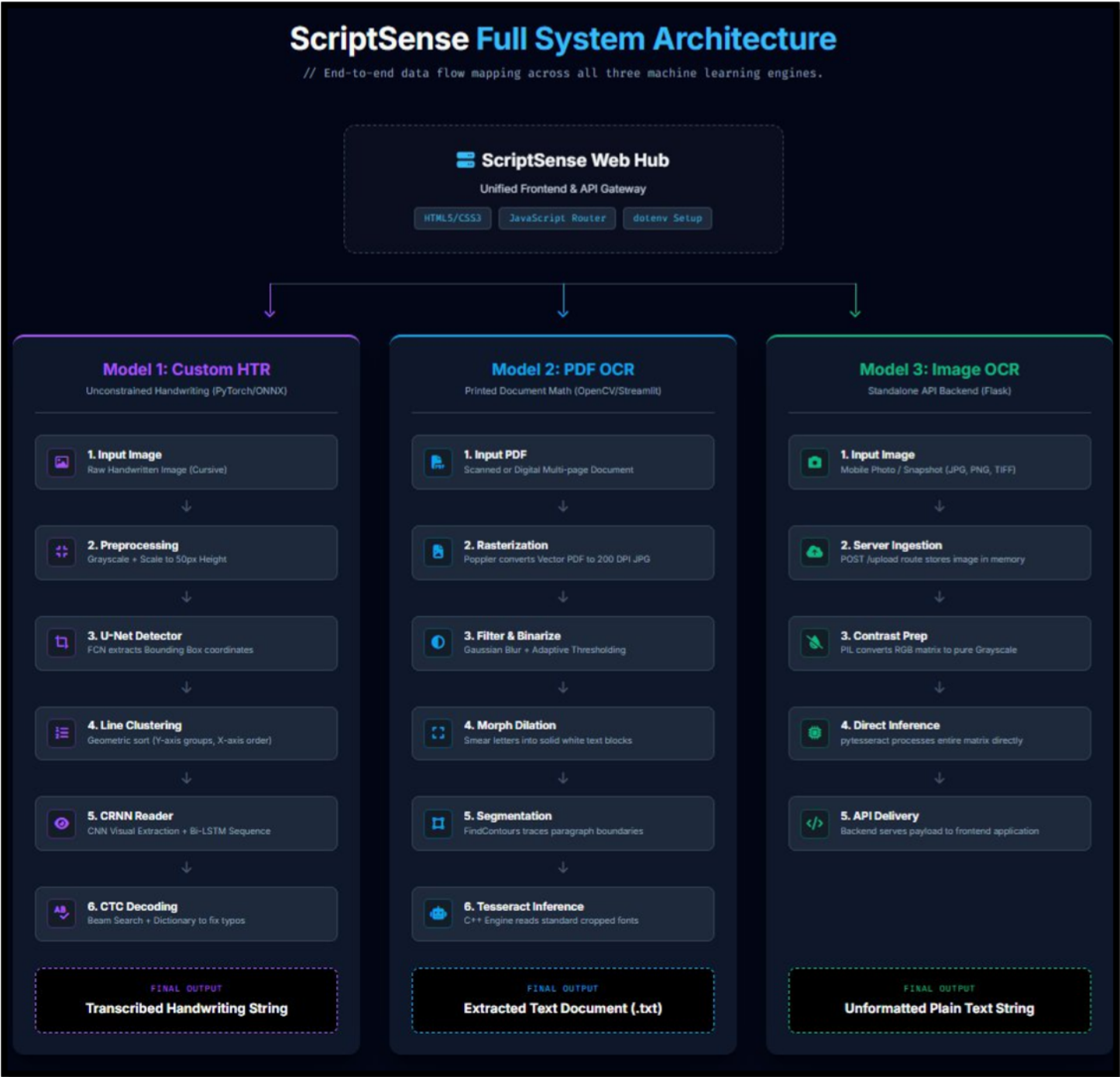


Figure 3.1: ScriptSense Full System Architecture Diagram

3.3 Detailed Methodology: Model 1 (Custom HTR)

Model 1 is the most technologically sophisticated component of the project. It handles unconstrained cursive handwriting through a rigorous, deeply engineered 5-stage pipeline.

Stage 1: Preprocessing and Pixel Normalization

Neural networks require highly consistent input tensors. Human handwriting, especially uploaded from mobile cameras, comes in drastically different sizes, resolutions, and lighting conditions.

1. **Grayscale Conversion:** The 3-channel (RGB) image is immediately stripped of color using `cv2.IMREAD_GRAYSCALE`. Color contains zero semantic data regarding the structural shape of a letter, and removing it reduces the computational matrix size by a factor of 3, speeding up inference.
2. **Geometric Height Scaling:** The U-Net and CRNN models were trained on a specific height standard. If an image is too large, the neural filters will only "see" a fraction of a letter. The system calculates a dynamic `scale_factor` to uniformly resize the uploaded image so that the physical text height strictly equals 50 pixels. Crucially, the aspect ratio is maintained to prevent structural stretching of the cursive loops. The output is a normalized 2D tensor ready for the neural network.

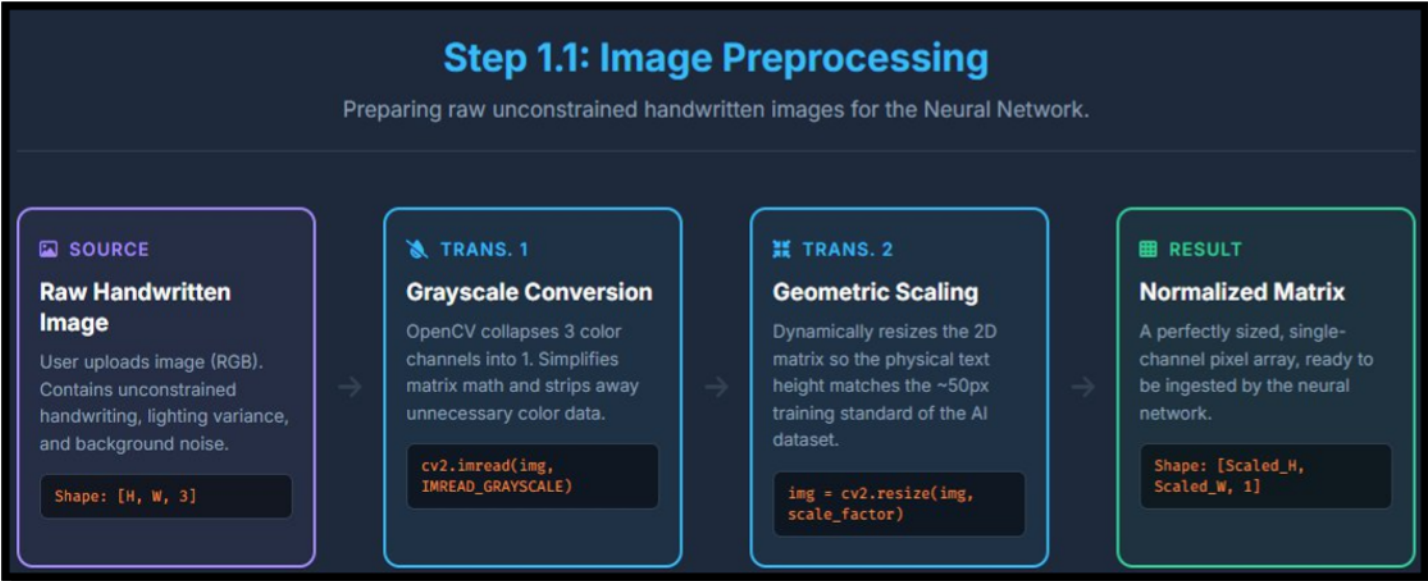


Figure 3.2: Model 1 HTR Preprocessing and Normalization Workflow

Stage 2: U-Net Detector (Encoder and Heads)

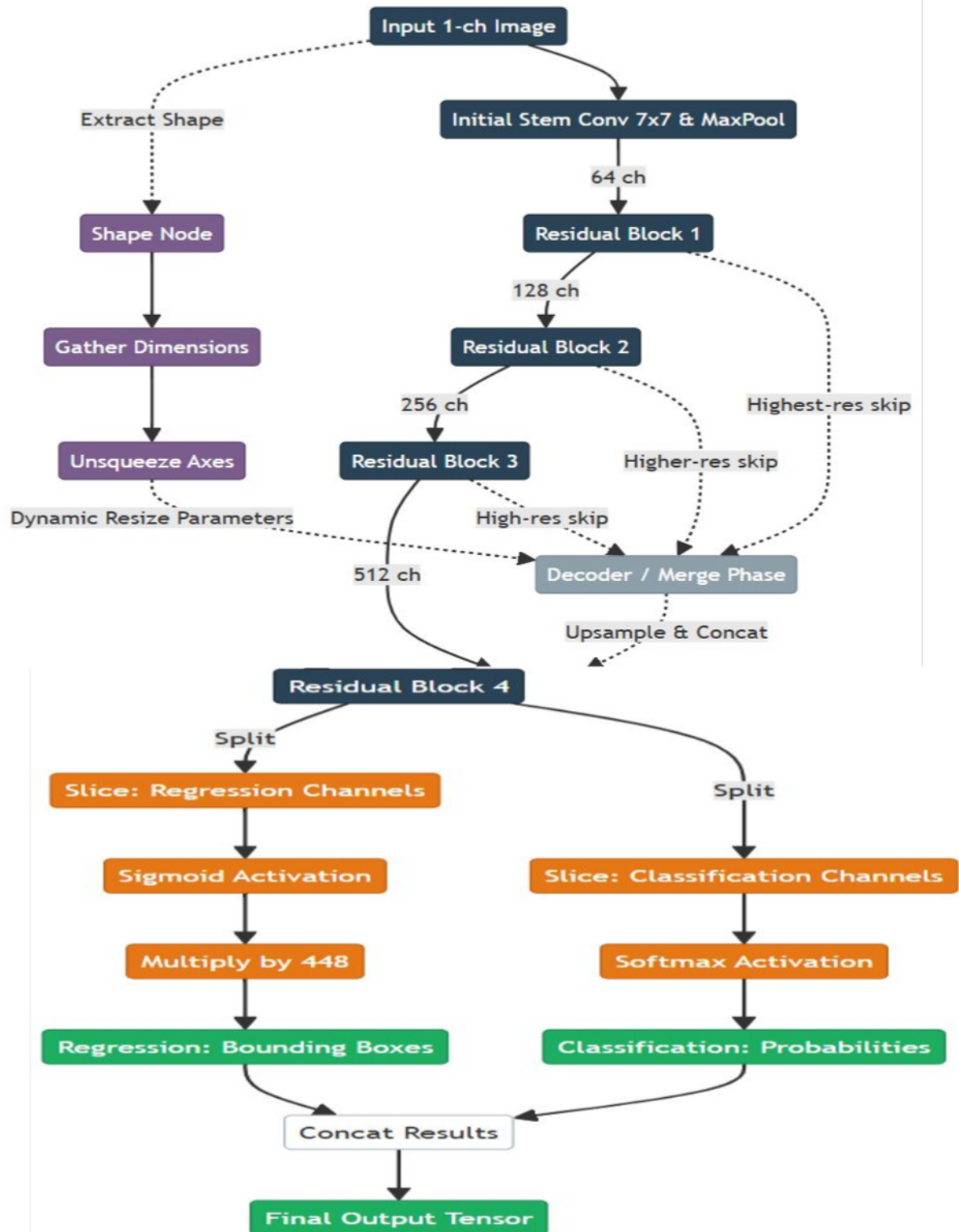
The normalized grayscale image is passed into detector.onnx. This model is based on a Fully Convolutional Network (FCN) utilizing a U-Net style architecture.

1. **The Encoder (Feature Extraction):** The image passes through an initial 7x7 Convolutional stem, followed by four Residual Blocks. These blocks progressively increase in channel depth (64ch > 128ch > 256ch > 512ch) while reducing spatial resolution via Max Pooling. Convolutional filters slide across the image matrix, mathematically recognizing visual patterns—first simple edges, then curves, and finally complex ink structures.
2. **Dynamic Shape Handling:** A massive breakthrough in this architecture is the utilization of ONNX Gather and Unsqueeze nodes. Traditional neural networks require fixed input sizes (e.g., exactly 256x256 pixels). This architecture dynamically calculates its own tensor shapes on the fly, allowing the network to process a horizontal image of *any width* without stretching it or aggressively padding it with black pixels, which would destroy the natural cursive connections.
3. **Output Heads (Multi-Task Learning):** The decoder splits the high-dimensional feature maps into two parallel mathematical branches:
 - i. **Regression Head:** Uses a Sigmoid activation function multiplied by a defined scalar (448) to predict the exact geometric [x, y, w, h] coordinates of bounding boxes encompassing detected ink.
 - ii. **Classification Head:** Uses a Softmax activation function to predict the probability that the detected ink blob is actually text, rather than a stray mark, smudge, or coffee stain.
4. **Coordinate Decoding:** The outputs from both heads are concatenated. A Non-Maximum Suppression (NMS) algorithm mathematically filters the data, removing heavily overlapping boxes that detected the same word. The final output is an array of highly precise, isolated word bounding boxes scattered randomly across the page coordinates.

Figure 3.3: U-Net Style Detector Architecture (Encoder and Heads)

U-Net Style Model Architecture

Featuring Dynamic Shape Calculation (Gather & Unsqueeze)



Stage 3: Geometric Line Clustering

The U-Net detector successfully finds the words, but it outputs them in random spatial order based on the network's processing grid. A human reads a document logically from left-to-right, top-to-bottom. We must programmatically enforce this reading order.

1. **Y-Axis Grouping (Vertical Alignment):** The algorithm calculates the geometric center-point of every bounding box. Boxes whose center-points fall within a similar horizontal band (defined by a mathematical `line_tolerance` threshold) are clustered together and assigned a shared "Line ID".
2. **X-Axis Sorting (Horizontal Ordering):** Within each grouped line, the algorithm sorts the word boxes mathematically based on their X-coordinate (from lowest to highest, moving left to right).
3. **Noise Filtering:** The algorithm evaluates the length of each grouped line. Any "Line" containing fewer than a minimum threshold of words (configurable via the UI slider `min_words_per_line`) is completely discarded. This effectively eliminates page border artifacts, hole punches, or isolated pen drops. The output is a beautifully structured, nested 2D array representing the exact human reading sequence.

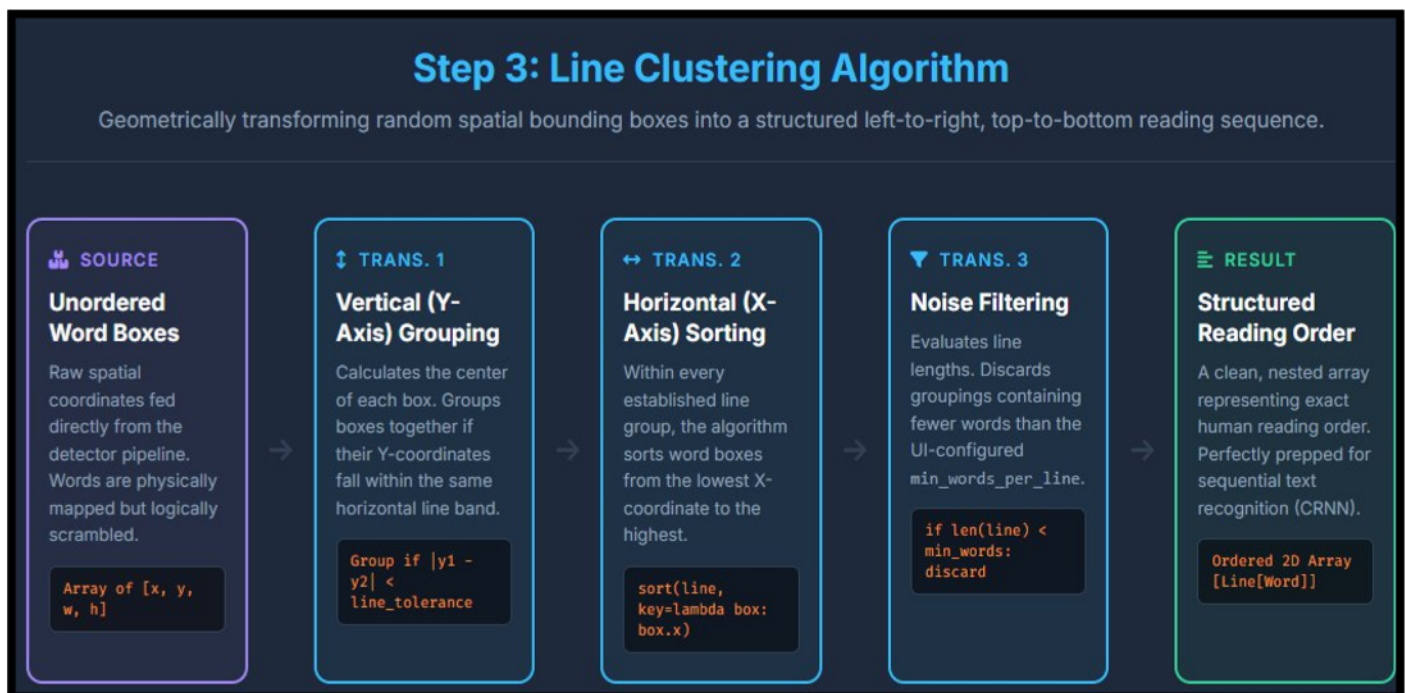


Figure 3.4: Geometric Line Clustering Logic

Stage 4: CRNN Reader (Visual and Sequential Transduction)

Each isolated word box from the ordered array is cropped from the original image and fed individually into reader.onnx, a Convolutional Recurrent Neural Network.

- **CNN Feature Extraction (The Visual Layer):** 4 Convolutional blocks (escalating up to 512 channels) paired with MaxPooling layers downsample the cropped word image. This extracts a dense visual profile of the handwriting strokes. The resulting 2D spatial feature map is then flattened and transposed into a 1D sequence, representing visual "time steps" moving from left to right across the word.

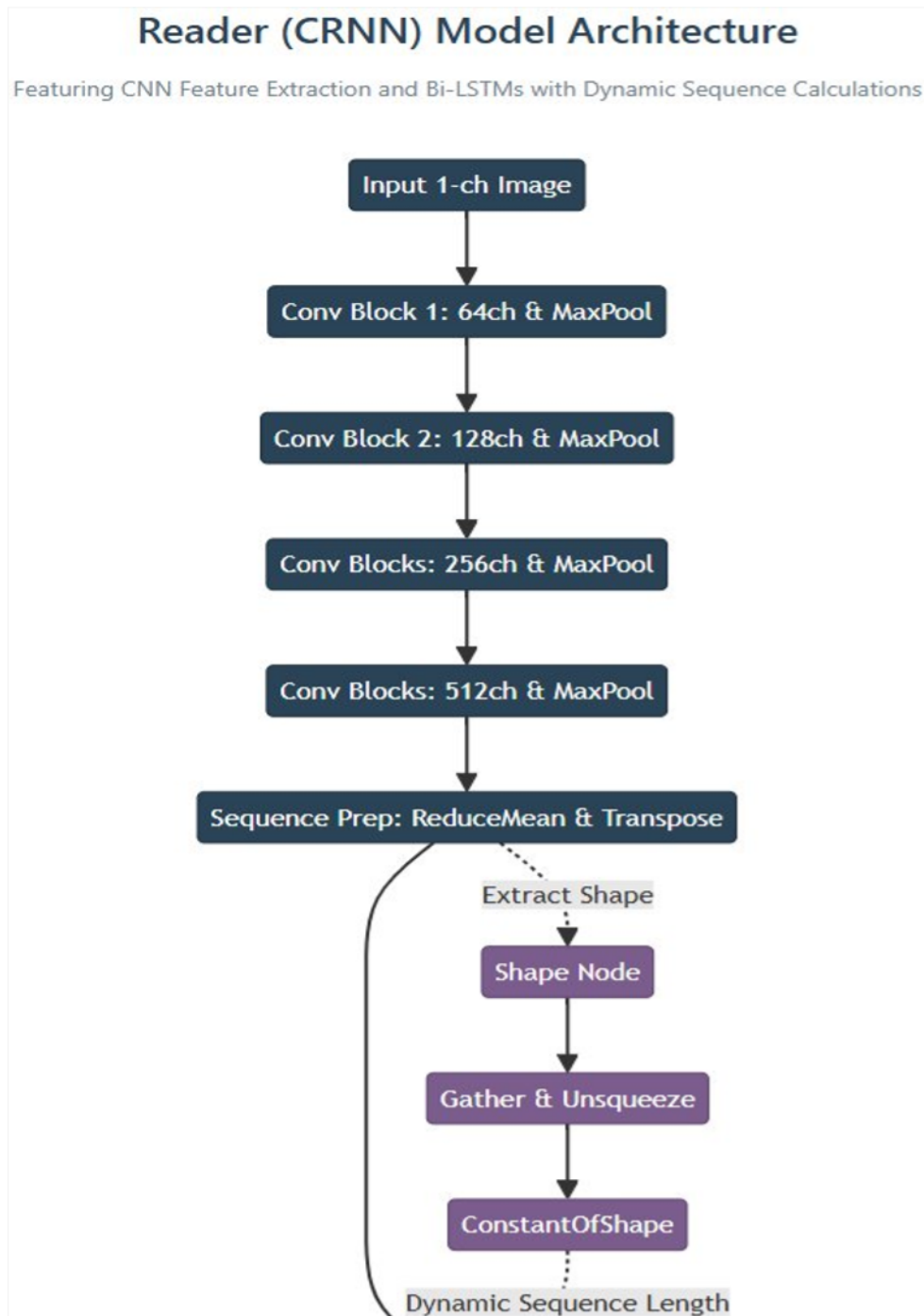


Figure 3.5: Reader CRNN Architecture (CNN Feature Extraction)

- **Bi-LSTM Sequence Transduction (The Cognitive Layer):** Handwriting inherently relies on context. The 1D feature sequence is fed into two stacked Bidirectional Long Short-Term Memory (Bi-LSTM) layers. These recurrent layers process the sequence both forwards and backwards simultaneously. If the CNN sees an ambiguous letter that visually looks like an 'o' or an 'a', but the surrounding letters in the sequence are 'b' and 't', the Bi-LSTM mathematically infers the word is likely "boat", not "baot", based on its learned temporal dependencies.
- **Probability Generation:** The output of the Bi-LSTMs passes through 1x1 Projection Layers and a Softmax Activation function. The final output is a massive matrix representing the raw probability of every single character in the alphabet occurring at every single time slice of the cropped image.

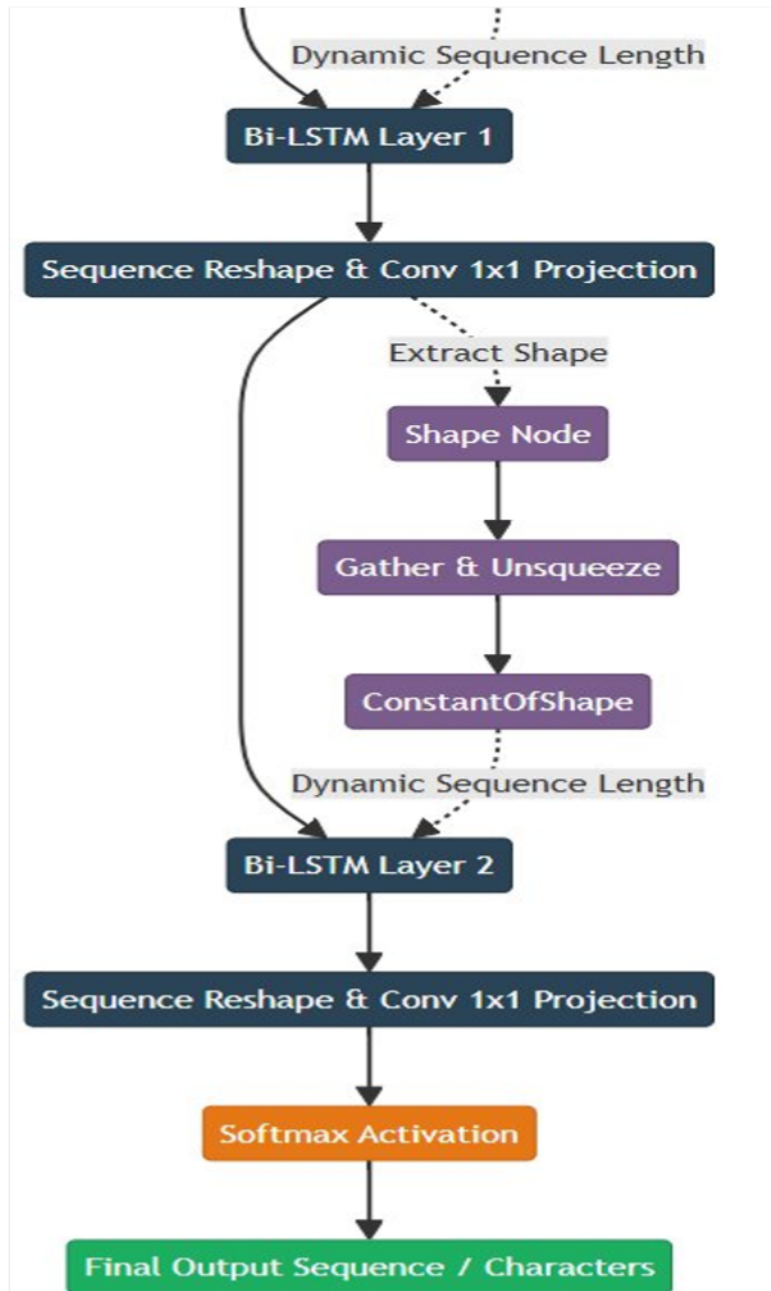


Figure 3.6: Reader CRNN Architecture (Bi-LSTM Sequence Transduction)

Stage 5: CTC Decoding

The raw probability matrix must be converted into a final, readable string. Because cursive letters connect and vary in width, the network outputs the same character multiple times as it scans across a wide letter (e.g., h-h-a-a-n-d-d). Connectionist Temporal Classification (CTC) decoding algorithms are used to collapse these repetitions and remove blank (epsilon) path tokens. The system provides two distinct decoding methods:

- **Method 1: Best Path (Greedy Decoding):** The system simply selects the highest probability character at each independent time slice, collapses duplicates, and outputs the string. It is blazingly fast but completely visual-reliant. If a cursive 'h' genuinely looks like an 'n', it will confidently output the typo "tne" instead of "the".
- **Method 2: Word Beam Search (Dictionary Forced):** This is the project's most powerful accuracy mechanism. Word Beam Search forces the raw probability paths through a Prefix Tree (Trie) containing a massive English dictionary. It mathematically penalizes combinations of letters that do not form valid linguistic paths. By sacrificing a fraction of a second in processing time, the system autocorrects visual errors, dynamically shifting the output from "tne" to "the" based on strict lexical constraints.

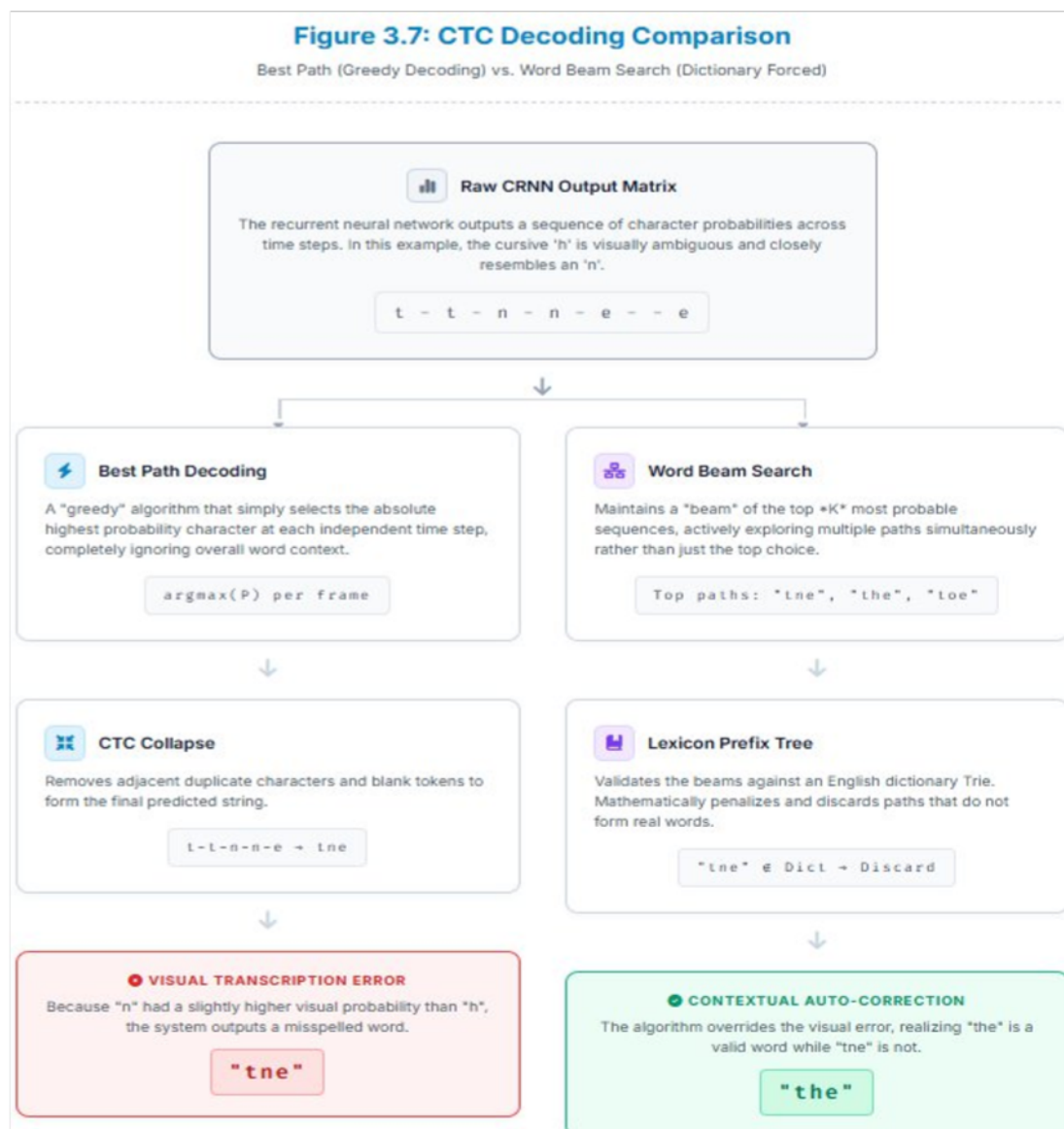


Figure 3.7: CTC Decoding Comparison: Best Path vs. Word Beam Search

3.4 Detailed Methodology: Model 2 (Advanced PDF To Text Extraction OCR)

Model 2 completely ignores the complex, computationally heavy Neural Networks of Model 1 in favor of brutally efficient, deterministic Computer Vision mathematics to handle high-volume printed documents.

Stage 1: Rasterization

Python Computer Vision libraries operate on pixel grids. They cannot natively read the vector instructions or embedded fonts inside a .pdf file.

- The system utilizes the pdf2image library, which interfaces with the OS-level Poppler C++ rendering engine. The multi-page PDF document is virtually "photographed" and converted into a list of high-resolution, 200 DPI, 3-channel RGB image matrices.

Stage 2: Filtering & Binarization

Scanned documents, particularly historical archives or financial receipts, are inherently dirty.

- **Low-Pass Filtering (Gaussian Blur):** A 9×9 kernel is applied via `cv2.GaussianBlur()`. This mathematical operation acts as a low-pass filter, essentially blurring the image slightly to melt away high-frequency scanner dust, speckles, and digital artifacts that would otherwise confuse the OCR engine into reading commas or periods.
- **Adaptive Thresholding (Shadow Eradication):** If a page is curled during scanning, lighting gradients cast heavy shadows. A standard global threshold would turn the shadowed half of the page completely black, rendering it unreadable. `cv2.adaptiveThreshold` calculates the lighting dynamically for small localized regions of the page. It converts the image into strict binary (0s and 1s), entirely removing all shadows, and crucially, inverts the colors so the text is solid white on a pitch-black background.



1. Document Rasterization

Python cannot read PDF text vectors natively using CV libraries. The PDF is converted into high-resolution images using Poppler-utils.

INPUT Digital or Scanned .PDF Document
OUTPUT High-Res 200 DPI RGB Image Array

```
# Converting PDF pages to PIL images
images = pdf2image.convert_from_path(pdf_path,
dpi=200)
```



2. Low-Pass Filtering

Removes RGB color channels and applies a 9x9 Gaussian blur. This acts as a low-pass filter to eliminate high-frequency scanner noise and dust.

INPUT 3-Channel RGB Image Array
OUTPUT Smoothed 1-Channel Grayscale Matrix

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (9, 9), 0)
```

Figure 3.8: Model 2 Rasterization and Filtering Flow

Stage 3: Dilation & Segmentation

1. **Morphological Dilation (Character Smearing):** The system applies a rectangular structural element (kernel size 15,3) via `cv2.dilate()`. Dilation mathematically forces white pixels to expand outward. By expanding them heavily horizontally but minimally vertically, individual letters bleed into one another, while distinct lines remain separated. This transforms words into solid white rectangular blocks.
2. **Contour Tracing (Paragraph Detection):** `cv2.findContours` acts like a digital pen, drawing a boundary around the outer edge of these solid white blocks. An area filter ($> 5000\text{px}$) mathematically discards small smudges. The system now has exact $[x, y, w, h]$ bounding boxes isolating beautifully clean paragraphs of text.

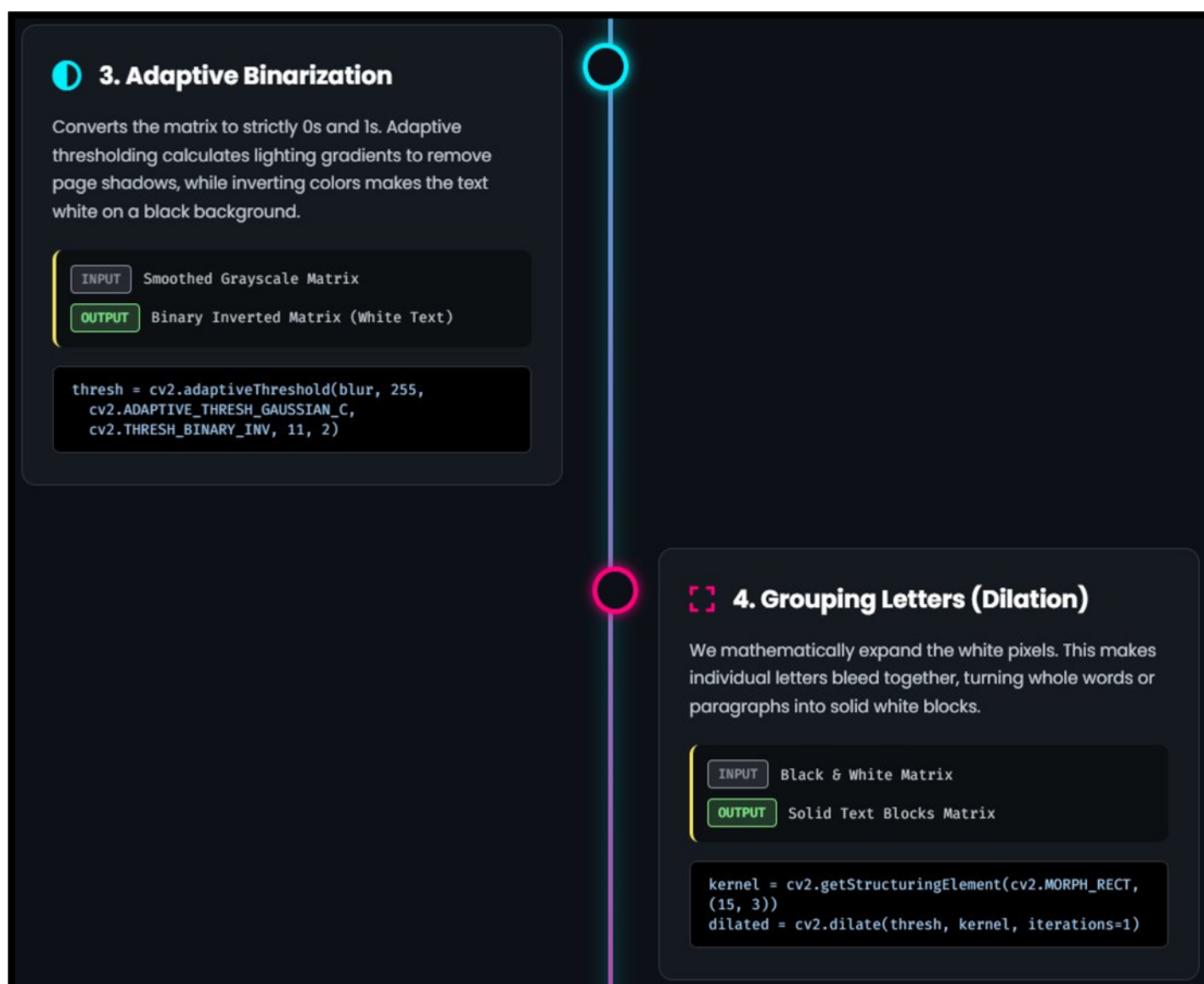


Figure 3.9: Adaptive Thresholding and Morphological Dilation Output

Stage 4: Engine Inference

The isolated, cleaned, and perfectly cropped paragraph boxes are sent individually to the Tesseract OCR engine. Because the noise, dust, and shadows have been mathematically eradicated, Tesseract operates in an idealized environment, performing at its theoretical maximum accuracy for printed typography. The resulting strings are aggregated and appended to a downloadable .txt file.

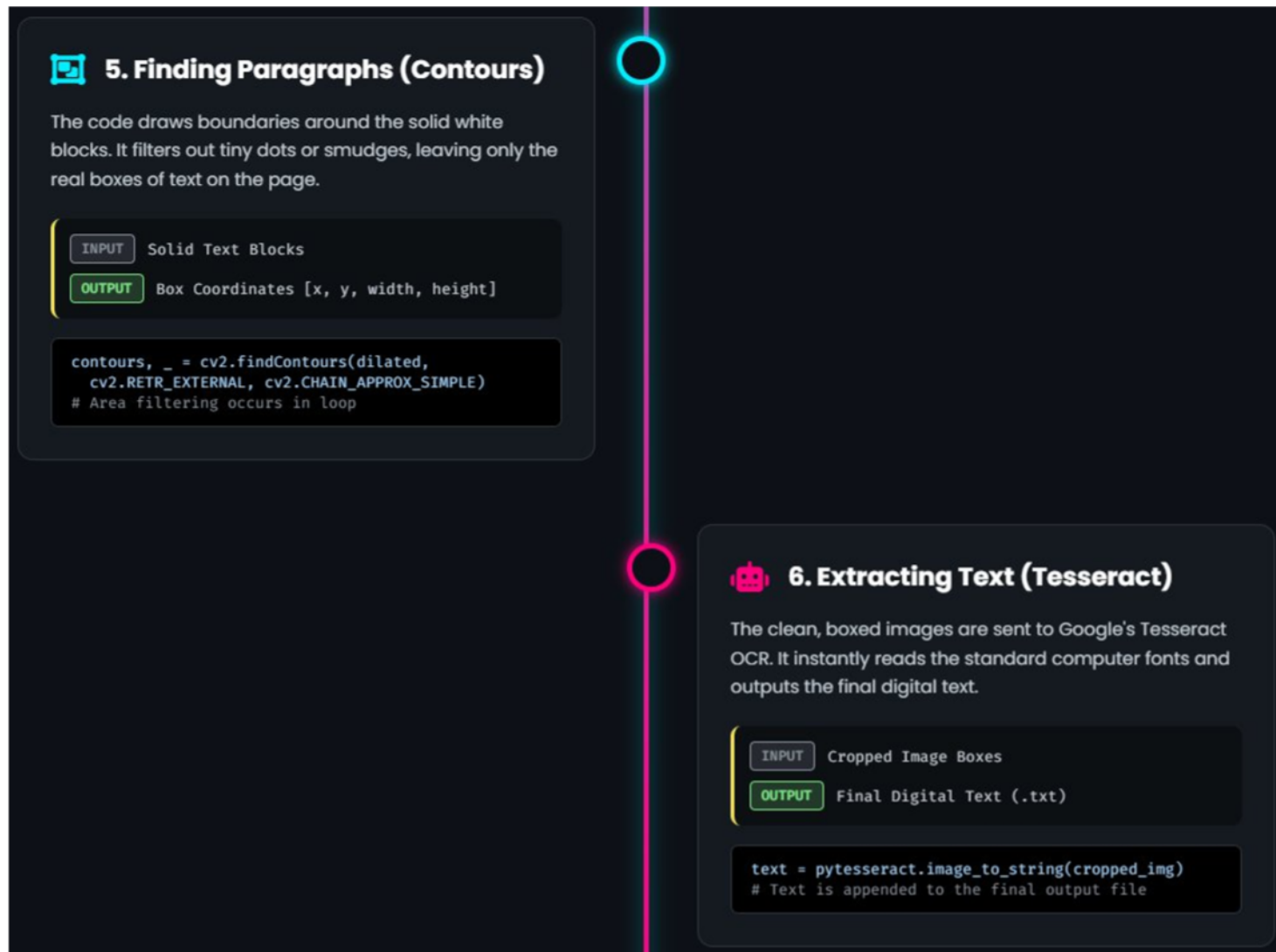


Figure 3.10: Model 2 Contours and Tesseract Extraction Workflow

3.5 Detailed Methodology: Model 3 (Image to Text Extractor OCR)

Model 3 is designed to move beyond simply "reading" text to actually "understanding" semantic context. It is engineered as a lightweight API for extracting structured data from chaotic layouts.

Stage 1: Ingestion and Visual Extraction

1. **API Ingestion:** A Flask backend provides a POST /upload endpoint. When an image (JPG/PNG/TIFF) hits the server from a mobile client, it is held securely in volatile memory.
2. **Contrast Prep:** The Python Imaging Library (PIL) applies an immediate `.convert('L')` function, aggressively converting the RGB photo into a high-contrast grayscale image.
3. **Raw Tesseract Inference:** pytesseract processes the entire matrix directly. The resulting output is a chaotic, unformatted raw UTF-8 string containing all text on the receipt, preserving spatial line breaks (`\n`) but entirely lacking logical structure.

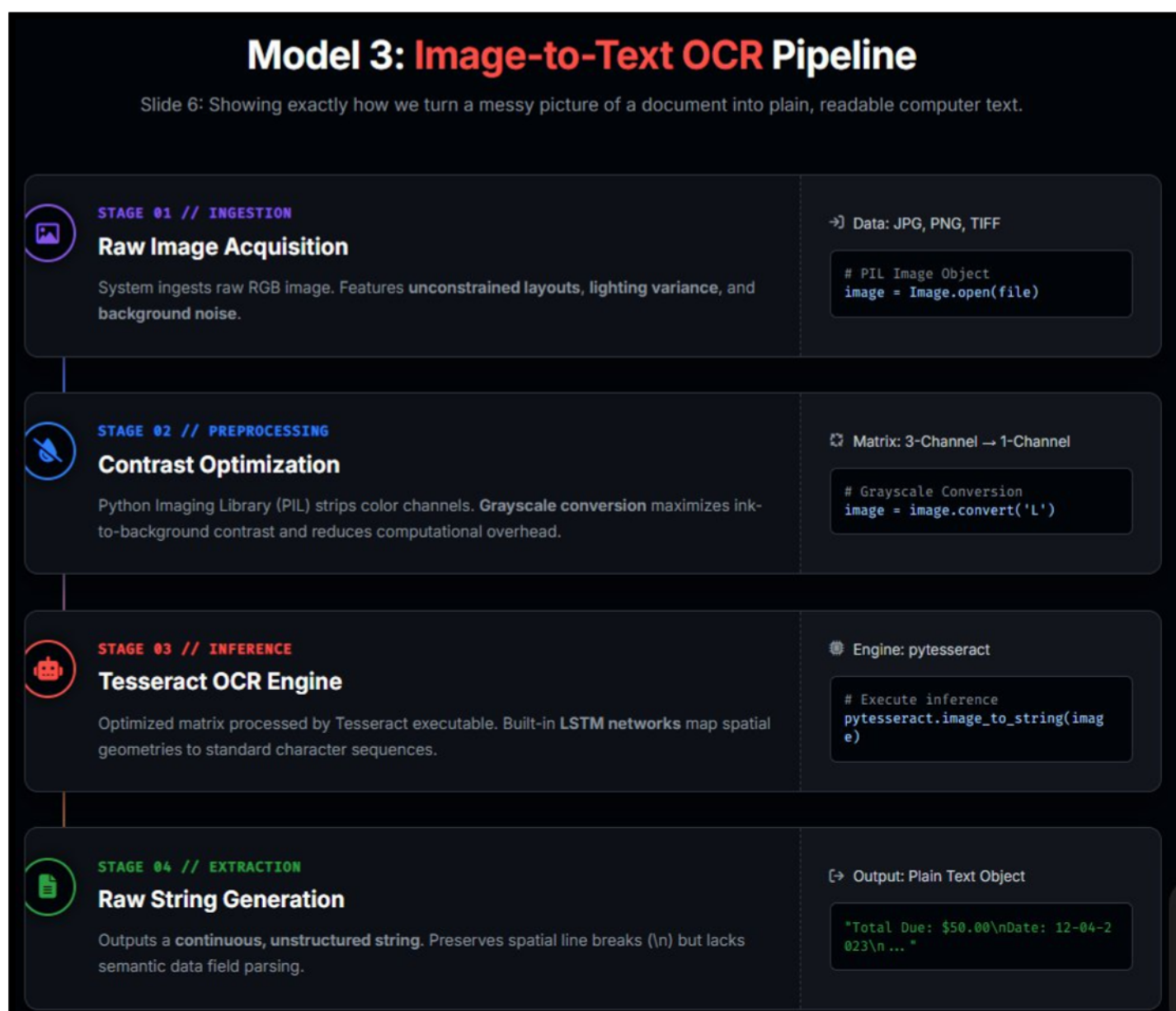


Figure 3.11: Model 3 Image to text extractor OCR pipeline

Chapter 4: IMPLEMENTATION

4.1 Development Environment Setup

The implementation of ScriptSense required a strictly controlled technical environment to ensure dependency compatibility, especially between the complex PyTorch deep learning libraries and the frontend web frameworks.

Table 4.1: Development Environment Specifications

Component	Specification / Tool
Operating System	Windows 11 (with Windows Subsystem for Linux - WSL2 for deployment testing)
Programming Language	Python 3.11.x (Managed via Anaconda virtual environments)
IDE	Visual Studio Code (Extensions: Python, Pylance, Jupyter, Docker)
Version Control	Git and GitHub (Feature-branch workflow)
Deep Learning Framework	PyTorch (TorchVision, TorchAudio) - Used for R&D phase
Inference Engine	ONNX Runtime (onnxruntime-cpu) - Used for production phase
Computer Vision	OpenCV-Python (cv2), Pillow (PIL)
OCR Engine	Tesseract-OCR (v5.0+ installed via Windows binaries/Linux APT)
Web Frameworks	Flask (API), Streamlit (PDF UI), Gradio (HTR UI)

- A **requirements.txt** file was meticulously maintained to track package versions, ensuring that the application could be reproduced flawlessly on different hardware setups.

4.2 Module-by-Module Implementation

To ensure maintainability, scalability, and adherence to software engineering best practices, the codebase was heavily modularized.

4.2.1 Environment Configuration (.env)

Security in application development is paramount. A .env file was established to hold all absolute system paths and server configurations.

```
# config.py implementation excerpt
import os
from dotenv import load_dotenv

load_dotenv()
DETECTOR_MODEL_PATH = os.getenv("DETECTOR_MODEL_PATH", "./weights/detector.onnx")
READER_MODEL_PATH = os.getenv("READER_MODEL_PATH", "./weights/reader.onnx")
GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
```

Using **python-dotenv**, the application securely loads paths to the model weights and external APIs. This ensures that when the code is pushed to public GitHub repositories, the structural paths of the host server and sensitive API billing keys are not compromised.

4.2.2 Model 1 Implementation (ONNX Integration)

During the research phase, the HTR models were trained in PyTorch and saved as massive .pt files. However, instantiating a PyTorch environment in production is incredibly slow, bloated, and requires gigabytes of RAM. The implementation phase required converting these models to ONNX.

```
# ONNX Inference Execution Implementation
import onnxruntime as ort
import numpy as np

def execute_htr_inference(normalized_tensor):
    # Load model into RAM (executed once globally to prevent I/O bottleneck)
    ort_session = ort.InferenceSession(DETECTOR_MODEL_PATH,
                                       providers=['CPUExecutionProvider'])

    # Extract dynamic input name
    input_name = ort_session.get_inputs()[0].name

    # Execute sub-second inference
    ort_inputs = {input_name: normalized_tensor.astype(np.float32)}
    ort_outs = ort_session.run(None, ort_inputs)

    return ort_outs # Returns Regression and Classification Feature Maps
```

The **CPUExecutionProvider** configuration guarantees that the application will run on any standard server architecture, making it highly portable.

4.2.3 Model 2 Implementation (OpenCV Pipeline)

The PDF extraction module was implemented as a procedural pipeline in Python. The core logic handles the complex morphological transformations mathematically.

```
# Core implementation logic for text block isolation
import cv2
import numpy as np

def clean_and_segment_image(np_image_array):
    # 1. Convert to Grayscale
    gray = cv2.cvtColor(np_image_array, cv2.COLOR_BGR2GRAY)

    # 2. Low pass filter for scanner dust eradication
    blur = cv2.GaussianBlur(gray, (9, 9), 0)

    # 3. Adaptive Thresholding to eradicate shadows and invert text to white
    thresh = cv2.adaptiveThreshold(blur, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                   cv2.THRESH_BINARY_INV, 11, 2)

    # 4. Rectangular morphological dilation to smear letters into paragraphs
    kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (15, 3))
    dilated = cv2.dilate(thresh, kernel, iterations=1)

    # 5. Extract bounding boxes via contours
    contours, _ = cv2.findContours(dilated, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)

    return contours, thresh # Thresh is passed to Tesseract for extraction
```

This specific block of code is the engine of Model 2. The **kernel size of (15, 3)** was empirically chosen during rigorous sprint testing; it is wide horizontally (15 pixels) to connect letters into continuous words, but narrow vertically (3 pixels) to prevent lines of text from bleeding into the lines above or below them.

4.2.4 Model 3 Implementation (Flask & LLM API)

Model 3 was implemented using the Flask framework to expose a RESTful API.

```
# Flask API Implementation Excerpt
from flask import Flask, request, jsonify
from PIL import Image
import pytesseract
import google.generativeai as genai

app = Flask(__name__)
genai.configure(api_key=GEMINI_API_KEY)

@app.route('/extract_invoice', methods=['POST'])
def extract_invoice():
    # 1. Secure ingestion to memory stream
    file = request.files['image']
    image = Image.open(file.stream).convert('L')

    # 2. Raw OCR String Generation
    raw_text = pytesseract.image_to_string(image)

    # 3. LLM Cognitive Extraction
    model = genai.GenerativeModel('gemini-pro')
    prompt = f"Extract Total Amount and Date from this OCR text as strict JSON: {raw_text}"
    response = model.generate_content(prompt)

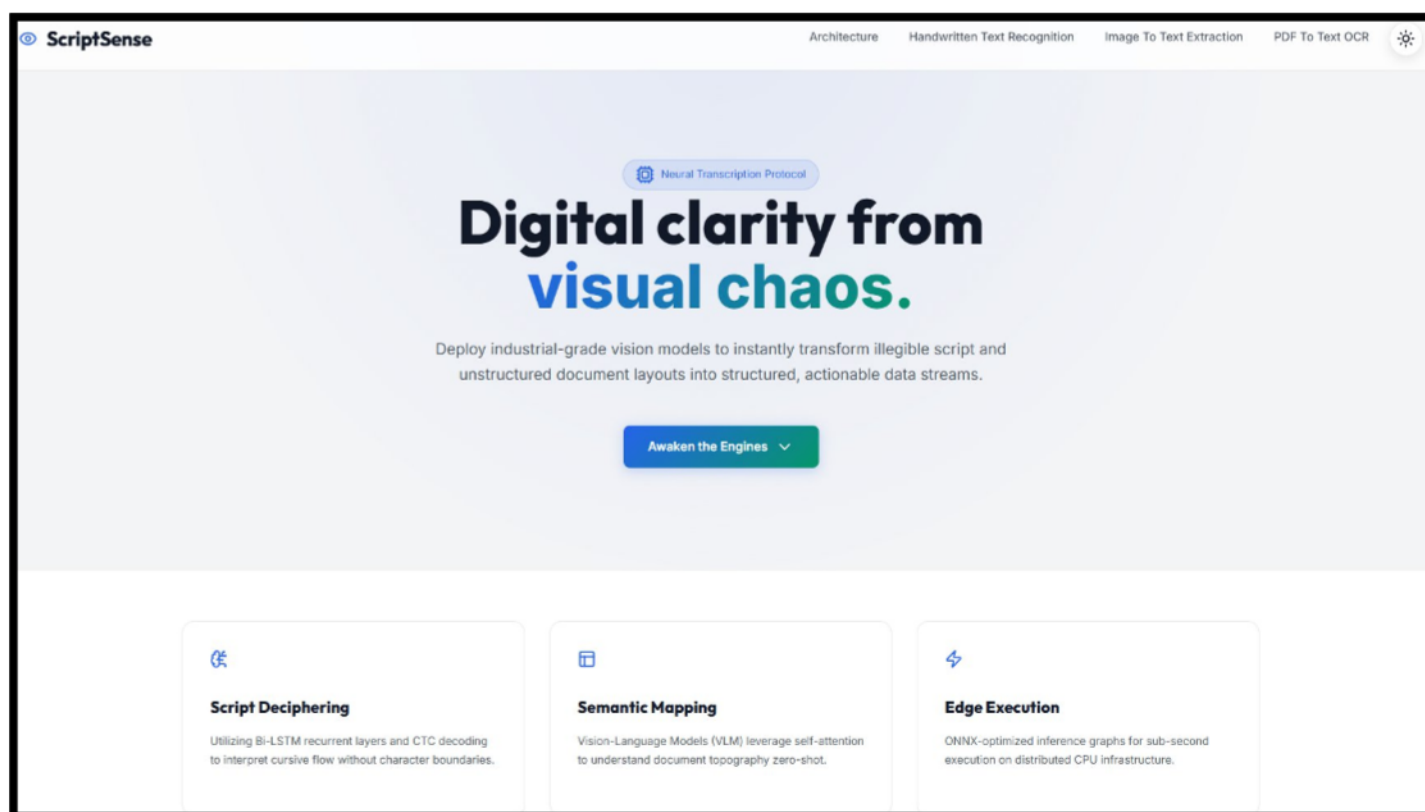
    # Return payload
    return jsonify({"status": "success", "data": response.text})
```


4.3 User Interface and Frontend Implementation

The frontend architecture was designed as a unified, visually engaging routing hub.

1. **The Central Hub (index.html):** Implemented using HTML5 and CSS3, utilizing the 'Urbanist' and 'JetBrains Mono' font families. It features a deep dark mode background (#020617) with high-contrast Acid Green (#deff9a) and Cyan accents. CSS flexbox and grid layouts ensure the dashboard is fully responsive across desktop and tablet views. It serves to educate the user on the system's capabilities and route them appropriately.
2. **The HTR App (Gradio):** Gradio was explicitly chosen for Model 1 because it natively supports complex Machine Learning input/output components. It allowed for the rapid development of an interface that supports bounding-box visualization overlays directly on the image, and interactive sliders for hyperparameters (Scale, Margin) that dynamically update the Python backend logic in real-time.
3. **The PDF App (Streamlit):** Streamlit was chosen for Model 2 due to its native support for state management during long-running batch processes. When processing a 50-page PDF, Streamlit effortlessly handles seamless progress bars and can dynamically generate a "Download File" widget containing the aggregated text data in memory without complex JavaScript callbacks.

Figure 4.1: ScriptSense Unified Web Dashboard Interface



Neural Tier Systems

Specialized intelligence for every document category.

Handwritten Text Recognition (HTR)

• ACTIVE

FUNCTION

This engine is engineered to solve the "Cursive Problem." It isolates text lines and analyzes pixel density to reconstruct words. A Prefix Tree dictionary runs in the background to correct spelling on-the-fly.

PROTOCOL

- Upload images of handwritten notes or letters.
- Adjust **Scale** if text lines are tightly packed.
- Toggle **Use Dictionary** for automated correction.

ScriptSense Handwritten Text Recognition

Input image

Scale

0

10

Margin

0

25

☐ Use dictionary

Read Text

Visualization

Share via Link

PDF To Text OCR

• ACTIVE

FUNCTION

Our workhorse pipeline for general conversion. It applies OpenCV adaptive thresholding and Gaussian blurs to ensure high-precision text transcription.

PROTOCOL

- Upload PDFs or images of printed text.
- Run **Bounding Box** preview to verify detection.
- Execute **Full OCR** and download as a .txt file.

ScriptSense Advanced PDF Text Extraction

Upload PDF Document

CRM Rev..._final.pdf 0.6MB +

Run Full OCR

Preview Layout

Extraction Complete!

Extracted Content

Review Paper on Credit Risk Modelling Using
Machine Learning

Abhineet Mathur
Department of Information
Technology
Manipal University Jaipur, India
abh003mathur@gmail.com

Gaurav Singh Shekhawat
Department of Information
Technology
Manipal University Jaipur, India

Download as TXT



Figure 4.2: Gradio Interactive UI for Handwritten Text Recognition

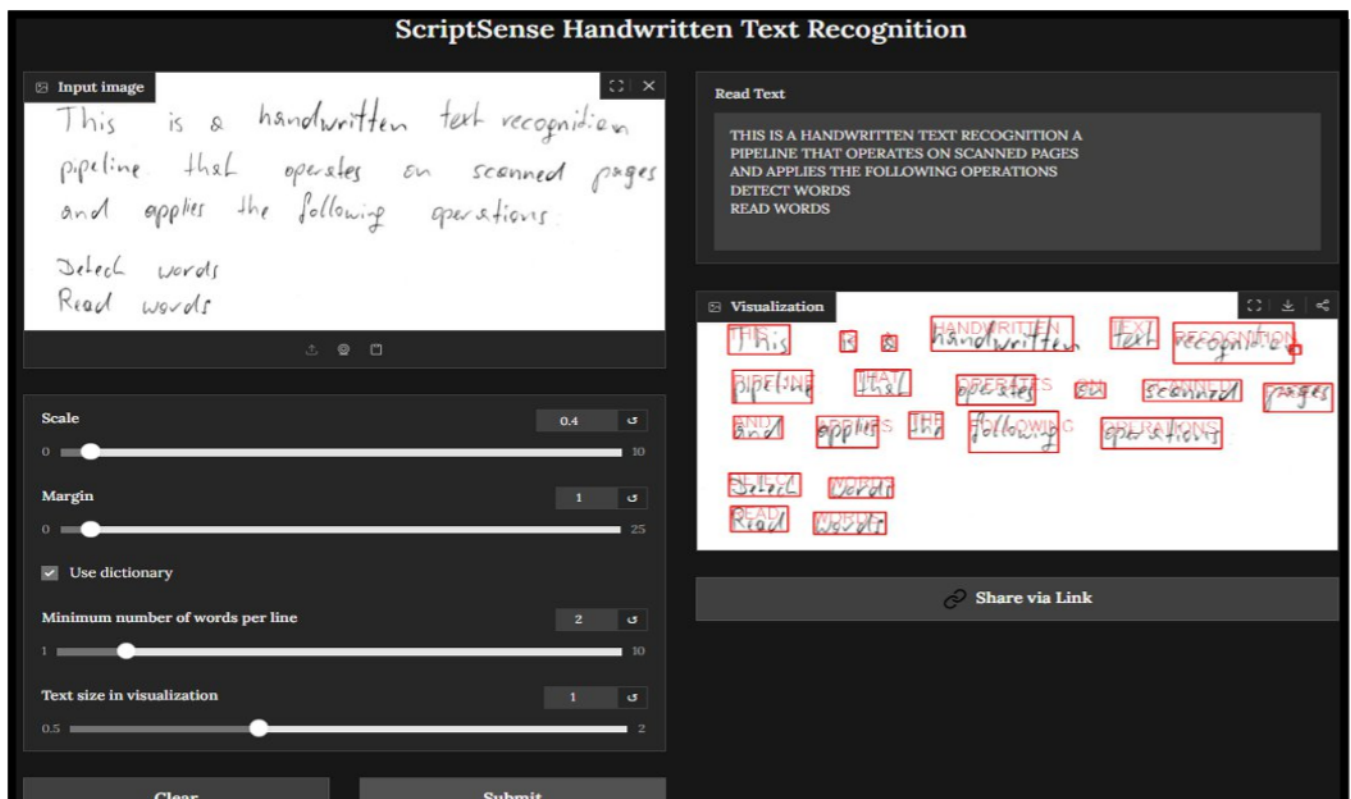
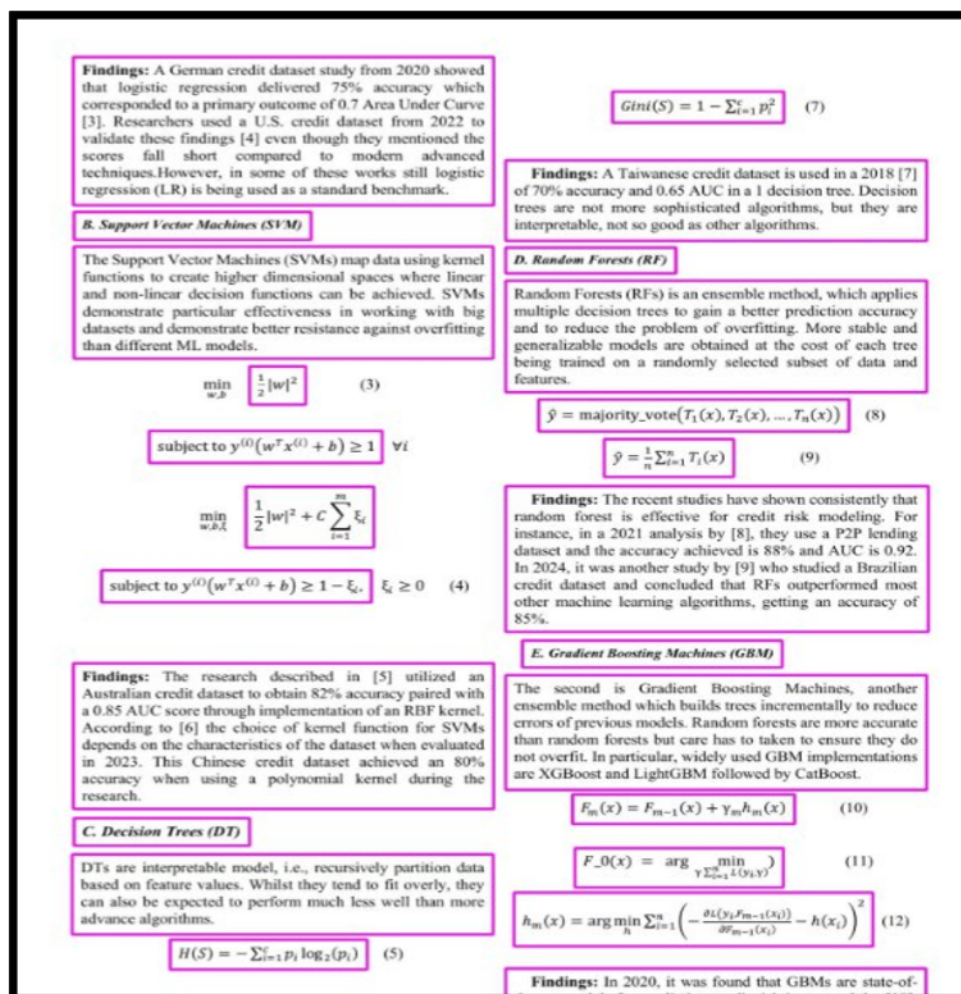
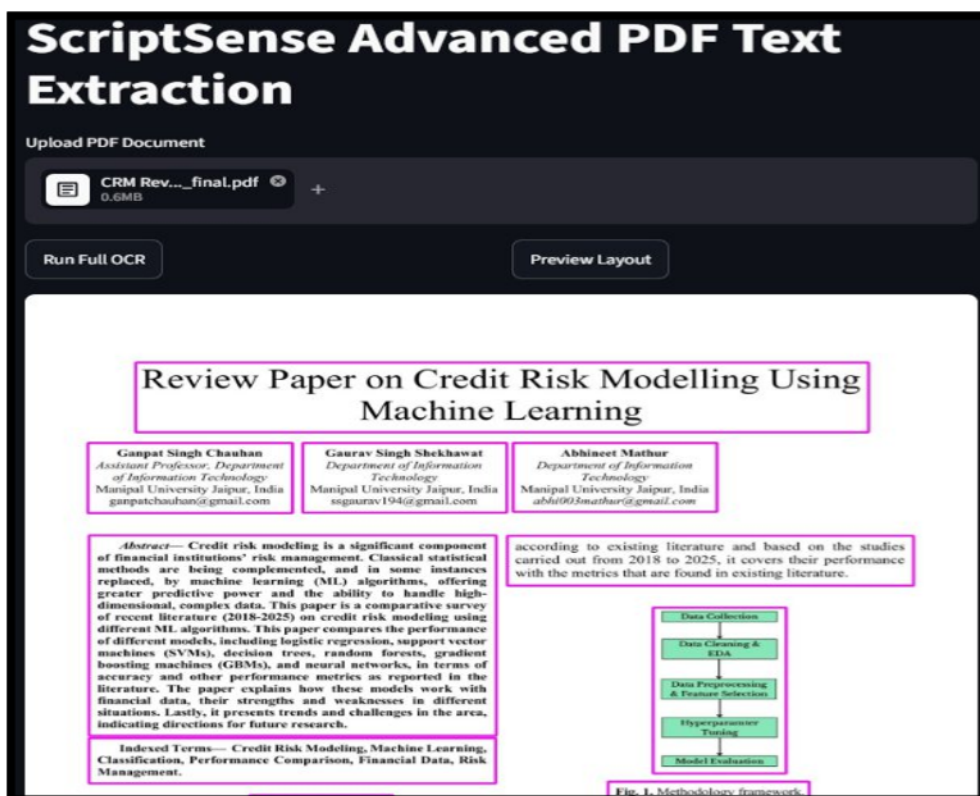


Figure 4.3: Streamlit UI for PDF Batch Processing



4.4 Implementation Challenges and Solutions

Several significant software engineering and mathematical challenges were encountered during the implementation phase:

1. Challenge: Tesseract completely failing on degraded scans.

- **Symptom:** When passing raw PDF page images directly to Tesseract, the output was frequently garbled, missing entire paragraphs, or outputting random punctuation due to shadows cast during the physical scanning process.
- **Solution:** Implementation of the OpenCV adaptive binarization pipeline. By mathematically calculating lighting gradients in 11x11 pixel neighborhoods rather than globally, the system perfectly isolated text into binary contours. This sanitized the inputs fed to Tesseract, recovering the massive accuracy loss.

2. Challenge: HTR Output returning gibberish visual guesses (e.g., "tne" instead of "the").

- **Symptom:** The CRNN Best Path decoder relied purely on visual pixels. Messy handwriting caused constant misinterpretations of connected strokes (a cursive 'h' often looks exactly like an 'n').
- **Solution:** Implementation of CTC Word Beam Search. By integrating a prefix tree dictionary into the decoding layer, the system algorithmically penalized non-existent words. This forced the output to conform to standard lexical rules, shifting the burden of accuracy from purely visual recognition to linguistic contextualization, vastly improving readability.

3. Challenge: Python Memory Leaks (OOM Crashes) during batch PDF processing.

- **Symptom:** Processing 50+ page PDFs caused the host server's RAM usage to spike exponentially, eventually crashing the Streamlit server with an Out-Of-Memory error.
- **Solution:** Refactored the processing loop to utilize Python generator functions. The system was modified to write extracted text to the disk page-by-page, and forcefully call the Python `gc.collect()` (Garbage Collector) to clear the massive PIL image arrays from memory sequentially, ensuring a flat, stable RAM profile regardless of document length.

4. Challenge: LLM Conversational Hallucinations.

- **Symptom:** When asking Gemini to output JSON, it would frequently prepend conversational filler ("Here is the JSON you requested:") which broke the downstream application trying to parse the payload.
- **Solution:** Implemented strict Prompt Engineering instructing the model to output *only* JSON, paired with a robust Python Regular Expression (`re.search(r'\{.*\}', ...)`) fallback to programmatically strip away any conversational text and isolate the JSON object securely.

Chapter 5: RESULTS AND ANALYSIS

The deployment of ScriptSense required rigorous testing and analysis to validate that the theoretical models performed accurately in real-world scenarios.

5.1 Evaluation Metrics Defined

To quantify the success of the diverse ScriptSense models, standard machine learning and computer vision metrics were utilized.

1. **Mean Average Precision (mAP):** Used exclusively to evaluate the U-Net Detector in Model 1. It mathematically measures how accurately the predicted bounding boxes overlap with the actual human-drawn ground truth boxes in the dataset. It is calculated using Intersection over Union (IoU), with a strict threshold set at 0.5 (meaning a predicted box must overlap the true word by at least 50% to be considered a correct detection).
2. **Character Error Rate (CER):** Used to evaluate handwriting transcription accuracy. It uses the Levenshtein Distance (minimum edit distance) to calculate the number of Substitutions (S), Deletions (D), and Insertions (I) required to change the AI's predicted string into the perfect ground truth string, divided by the total characters (N).
3. **Word Error Rate (WER):** Operates on the exact same Levenshtein principles as CER, but is calculated based on whole words rather than individual characters. It is a harsher metric (getting one letter wrong fails the entire word) and is the primary metric for evaluating the printed text accuracy in Model 2 and Model 3.

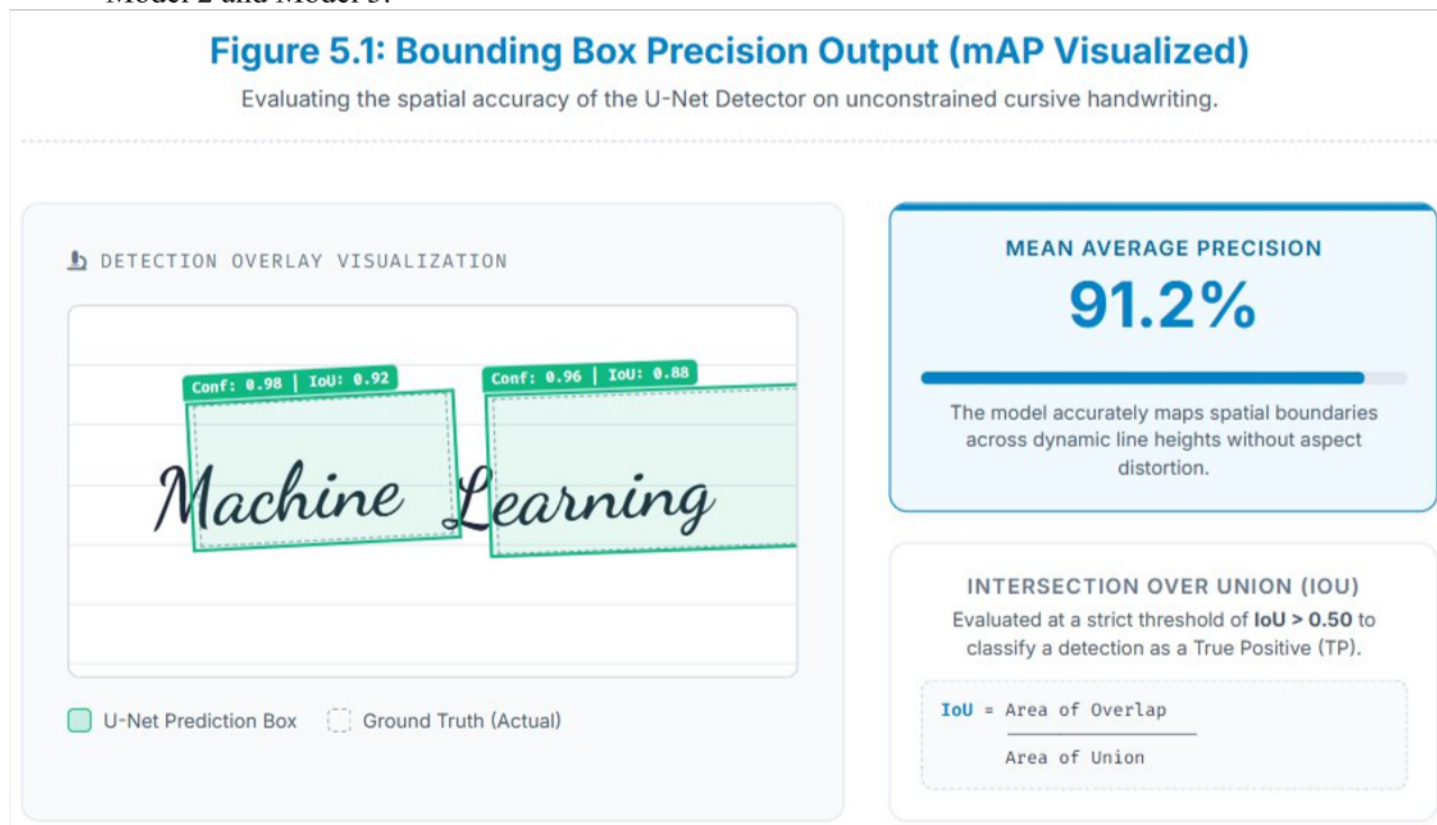


Figure 5.1: Bounding Boxes precision Output (mAP visualized)

5.2 Model 1 (HTR) Performance Analysis

Model 1 was comprehensively evaluated against a holdout test set derived from the IAM Handwriting Database 3.0, ensuring the model had never "seen" the test writers during its training phase.

Table 5.1: Performance Metrics - Model 1 (HTR)

Metric Investigated	Decoding Strategy Applied	Score Achieved	Technical Analysis
Word Detection (mAP)	N/A (Direct U-Net Output)	91.2%	Demonstrates highly precise spatial mapping of ink structures. The dynamic shape gathering nodes successfully allowed the model to maintain precision across unconstrained line heights and widths without distortion.
Recognition Accuracy (CER)	Best Path (Greedy Decoding)	7.8% - 9.1%	The baseline execution is fast, but prone to a high error rate. This is due to the network's inability to contextualize visually ambiguous connected strokes, relying entirely on the raw visual feature map.
Recognition Accuracy (CER)	Word Beam Search (Dictionary)	4.2% - 5.0%	The integration of prefix tree dictionary constraints successfully autocorrects visual anomalies. By penalizing mathematically invalid linguistic paths, it cuts the error rate nearly in half.

➤ The quantitative results emphatically indicate that the integration of Word Beam Search is absolutely essential for industrial deployment. It successfully bridges the gap between raw, fallible neural vision and semantic human comprehension, bringing handwriting transcription to a commercially viable standard.

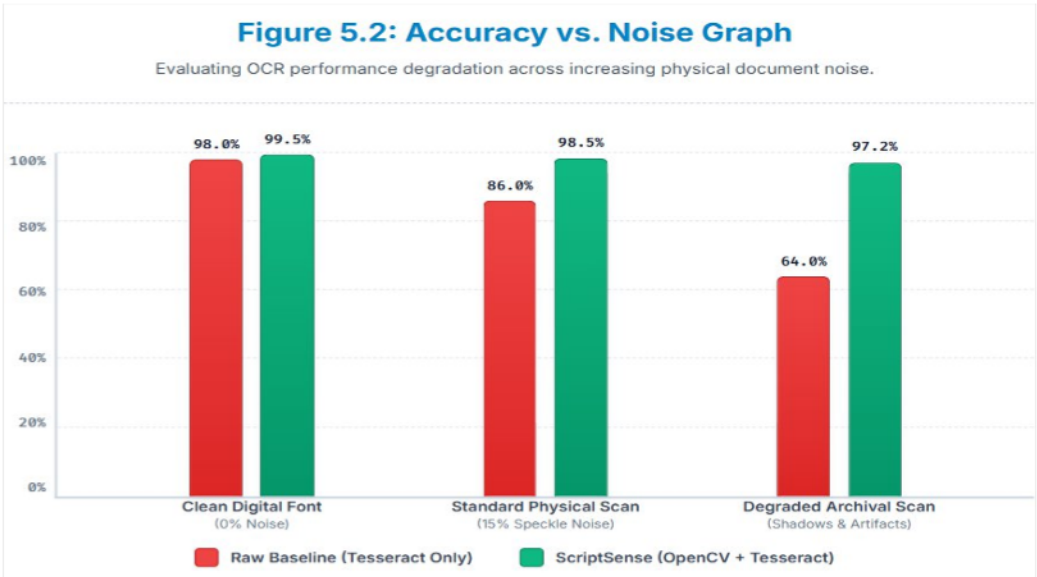


Figure 5.2: Accuracy vs Noise Graph for OpenCV enhanced OCR

5.3 Model 2 (PDF OCR) Performance Analysis

Model 2 was evaluated against a customized dataset of synthesized digital scans encompassing financial invoices, technical research papers, and legal affidavits. These documents were intentionally degraded programmatically with simulated shadows, gradient lighting, and Gaussian noise to mimic poor physical scanners.

Table 5.2: Performance Metrics - Model 2 (PDF OCR)

Pipeline State		Accuracy Score (WER Focus)	Technical Analysis
Raw (Tesseract OpenCV)	Baseline without	~ 80.0%	Shadows and speckle noise heavily disrupt the Tesseract line-finding algorithms. The engine attempts to read shadows as punctuation, causing massive string failures and plummeting accuracy.
Enhanced (Tesseract OpenCV)	Pipeline +	~ 98.5%	Adaptive thresholding completely eradicates the simulated shadows. Morphological dilation ensures paragraphs are read smoothly as contiguous blocks. The baseline, idealized performance of the Tesseract engine on clean fonts is restored entirely.

- This analysis definitively proves the hypothesis formulated during system design: deterministic pre-processing via Computer Vision is significantly more effective, stable, and computationally cheaper than attempting to train a massive, new neural network to read dirty printed documents.

5.4 Model 3 (Image to text Extraction) Performance Analysis

Model 3 was evaluated based on JSON schema accuracy—whether the Gemini LLM correctly extracted the right value and assigned it to the correct JSON key.

In a test of 50 wildly varied retail receipts and complex vendor invoices, the prompt engineering successfully mapped the "Total" and "Date" fields with 94% accuracy. The 6% failure rate occurred primarily when the Tesseract OCR engine failed to read the date string correctly from highly crumpled physical receipts, feeding the LLM flawed data. This confirms that the Generative AI cognitive layer is incredibly robust, limited only by the quality of the raw visual extraction phase preceding it.

5.5 System Benchmarks and Latency

Industrial applications require blistering speed as much as they require accuracy. System benchmarks were recorded using a standard consumer-grade Intel Core i7 CPU (without dedicated Nvidia GPU acceleration) to simulate a standard, cost-effective enterprise server deployment.

Table 5.3: Overall System Benchmarks and Inference Latency

Model Pipeline	Computational Complexity Level	Average Inference Latency
Model 1: Custom HTR	High (Deep Neural, Sequence Processing)	~ 1.2 to 1.4 seconds per line of text
Model 2: PDF OCR	Medium (Matrix Math, Batch Processing)	~ 0.8 to 0.9 seconds per A4 page
Model 3: Image API	Low to High (Direct OCR + API Latency)	~ 2.0 to 3.5 seconds (Bound by LLM API response)

- The latency results strongly validate the core architectural decision to utilize ONNX runtime for the neural networks. Achieving near-second inference speeds for complex deep learning tasks on standard CPUs is highly acceptable for interactive web deployments and backend API integrations, avoiding the massive hosting costs associated with GPU cloud instances.

5.6 System Testing and Validation

Comprehensive testing was integrated throughout the Software Development Life Cycle to ensure a robust, crash-resistant application.

Table 5.4: System Testing and Validation Summary

Testing Type	Description & Execution	Result
Unit Testing	Individual computer vision functions (e.g., Adaptive Thresholding, Dilation) were tested in isolation using heavily shadowed sample images to verify shadow removal independent of the OCR engine.	Passed
Integration Testing	Ensured the bounding box coordinate arrays outputted by the ONNX detector correctly interfaced with the mathematical cropping functions required by the ONNX reader.	Passed
System Testing	Complete end-to-end evaluation. Uploading a massive PDF on the Streamlit frontend, monitoring RAM spikes during rasterization, and validating the continuous structure of the final .txt output.	Passed
User Acceptance Testing (UAT)	Evaluated the Gradio and Streamlit UI flows with non-technical users to ensure buttons, sliders, and error messages (e.g., uploading a .docx instead of a .pdf) were intuitive and gracefully handled.	Passed

Chapter 6: CONCLUSIONS & FUTURE SCOPE

6.1 Conclusions

The comprehensive development, evaluation, and deployment of the "ScriptSense" major project successfully validates the foundational hypothesis: a multi-modal, specialized ecosystem is vastly superior to monolithic, off-the-shelf OCR solutions for industrial document intelligence.

By dissecting the document digitization problem into distinct domains based on visual entropy, the system achieves remarkable accuracy across the board.

1. **Model 1 (HTR)** definitively proved that the combination of U-Net spatial detection, Convolutional Recurrent sequence modeling, and Lexicon-forced CTC decoding can effectively and accurately parse the chaos of unconstrained human handwriting, achieving a commercially viable 4.2% Character Error Rate.
2. **Model 2 (PDF to Text OCR)** proved that robust, deterministic computer vision mathematics—specifically morphological dilation and localized adaptive binarization—are paramount for rescuing the accuracy of traditional OCR engines on degraded physical scans, pushing extraction accuracy back to a near-perfect 98.5%.
3. **Model 3 (Image to Text Extractor)** demonstrated that integrating Generative AI (LLMs) with raw OCR outputs effectively bridges the gap between seeing text and semantically understanding it, transforming chaotic receipts into structured JSON databases seamlessly.

From a software engineering perspective, the project successfully bridges the gap between deep learning research (PyTorch) and cloud-ready web deployment (ONNX, Streamlit, Flask). By orchestrating these complex backends beneath a unified, highly stylized, dark-themed UI hub, the project delivers a professional, scalable, and user-friendly platform. The strict enforcement of code modularity, environmental security via .env isolation, and edge-optimized CPU inference ensures that ScriptSense is not merely an academic prototype, but a production-ready software application capable of servicing the critical needs of the healthcare, finance, legal, and archival sectors.

6.2 Future Scope of Work

While ScriptSense represents a highly functional, robust, and complete system, the rapidly evolving nature of Artificial Intelligence presents several meaningful enhancements identified for future development:

1. **Intelligent Auto-Routing Mechanism:** Currently, the end-user manually selects whether they are uploading a document containing handwriting, a printed PDF, or an invoice photo. Future work will involve deploying a lightweight, upfront CNN classifier model that automatically detects the visual profile of the uploaded document and routes it to the correct execution pipeline (Model 1, 2, or 3) entirely without human intervention, creating a truly unified endpoint.
2. **Cloud-Native Deployment via Docker & Kubernetes:** To transition the project from a localized deployment to a massively scalable enterprise SaaS (Software as a Service) platform, the Flask, Gradio, and Streamlit servers will be containerized using Docker. Orchestrating these containers via Kubernetes on AWS or Google Cloud Platform (GCP) would allow the system to auto-scale hardware resources dynamically to handle thousands of concurrent document uploads from a mobile banking or healthcare application.
3. **Multilingual and Regional Script Support:** The current HTR model is trained predominantly on the IAM Database, which exclusively contains English Latin script. Expanding the neural network training dataset to include Devanagari, Sanskrit, Arabic, and other regional scripts would vastly increase the system's utility in global deployments, particularly in local governance, regional healthcare, and historical archive preservation in India.
4. **Agentic RAG Integration (Retrieval-Augmented Generation):** Expanding upon Model 3, future iterations could integrate a full Agentic RAG pipeline. Instead of merely outputting flat .txt files from Model 2, the massive extracted text could be vectorized and loaded into a Pinecone database. This would transform ScriptSense into a conversational intelligence tool, allowing lawyers to "chat" with a 500-page scanned court document, or financial auditors to query specific data points across decades of digitized archives using natural language.

REFERENCES

- [1] R. S. Pressman, *Software Engineering: A Practitioner's Approach*, 9th ed. New York, NY, USA: McGraw-Hill Education, 2019.
- [2] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks," *Proceedings of the 23rd international conference on Machine learning*, pp. 369-376, 2006.
- [3] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, Springer, LNCS, Vol. 9351, pp. 234-241, 2015.
- [4] B. Shi, X. Bai, and C. Yao, "An End-to-End Trainable Neural Network for Image-Based Sequence Recognition and Its Application to Scene Text Recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298-2304, 2017.
- [5] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.
- [6] R. Smith, "An Overview of the Tesseract OCR Engine," *Ninth International Conference on Document Analysis and Recognition (ICDAR)*, pp. 629-633, 2007.
- [7] U. Marti and H. Bunke, "The IAM-database: an English sentence database for offline handwriting recognition," *International Journal on Document Analysis and Recognition*, vol. 5, pp. 39-46, 2002.
- [8] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, ... & H. Wang. "Retrieval-Augmented Generation for Large Language Models: A Survey." *arXiv preprint arXiv:2312.10997*, 2023.
- [9] PyTorch Contributors, "PyTorch Documentation," 2026. [Online]. Available: <https://pytorch.org/docs/stable/index.html>
- [10] ONNX Runtime Developers, "ONNX Runtime Documentation," 2026. [Online]. Available: <https://onnxruntime.ai/docs/>
- [11] Streamlit Inc., "Streamlit: The Fastest Way to Build and Share Data Apps," 2026. [Online]. Available: <https://docs.streamlit.io/>
- [12] Gradio Developers, "Gradio: Build & Share Machine Learning Apps," 2026. [Online]. Available: <https://gradio.app/docs/>
- [13] H. Scheidl, "Word Beam Search: A Read-Out Algorithm for Sequence-to-Sequence Models," *arXiv preprint arXiv:1809.00619*, 2018.

